IE University

School of Science & Technology

Hierarchical N-Body Simulation of Galactic Dynamics in WebGPU

Enabling Scalable and Interactive Physics Simulations on Modern Web Platforms

Zaid Alsaheb

Bachelor of Computer Science & Artificial Intelligence

Supervised by: Professor Raul Perez Pelaez

Abstract

WebGPU is a recent cross-platform GPU standard that promises native-level compute performance in both desktop and browser environments, but it has seen little use for scientific workloads. We test that promise by implementing a complete Barnes–Hut gravitational N-body solver and measuring where the abstraction costs actually fall. Our implementation builds a Linear Bounding Volume Hierarchy on the GPU every timestep using Karras's parallel method, evaluates forces via BVH traversal, and integrates with a symplectic leapfrog, all from a single C++/WGSL codebase that compiles to both native desktop and browser targets. On an Apple M2 Macbook Air, we benchmark four WebGPU implementations (wgpu-native, Dawn, Chrome, Safari) and compare against a native Metal Barnes–Hut solver. The WebGPU abstraction costs 2.0× at $N = 1000$ but outperforms the Metal baseline above $N = 5000$, reaching 2.9× faster at $N = 100000$ thanks to a fully GPU-resident tree pipeline. Force evaluation dominates step time (94–99%); LBVH construction stays below 0.35 ms regardless of particle count. In the browser, Chrome is competitive at small N but scales worse, settling at a 1.4 × overhead by $N = 100000$. Numerical fidelity is limited by both 32-bit precision and the tree force approximation. These results show that WebGPU can support interactive galactic dynamics simulation — including parallel tree construction and hierarchical force evaluation — without specialised hardware or vendor lock-in.

1) Literature Review

Galaxy simulation boils down to computing gravitational forces among many particles and stepping them forward in time without the numerics falling apart. GPU-accelerated N-body solvers are well established on platforms like CUDA, but whether the newer, cross-platform WebGPU standard can handle the same workload and what it would cost in practice, is largely untested. This review covers the four areas that bear on that question: N-body force algorithms, numerical integration, GPU-accelerated computing, and browser-based GPU computation.

Computational Constraints in Astrophysical Simulation

For most of the field's history, computational astrophysics could rely on Moore's Law: implement an algorithm once and wait for faster hardware. Sutter documented the end of that era in 2005, noting that single-core clock speeds had hit a wall (Sutter, 2005). Since then, progress has meant finding parallelism by writing code that can split work across hundreds or thousands of execution units rather than waiting for any single one to get faster (Fluke et al., 2011).

GPUs are the clearest beneficiary of that shift. Their thousands of lightweight cores and high memory bandwidth can accelerate suitable workloads by an order of magnitude or more (Owens et al., 2007). But "suitable" is doing real work in that sentence, as an algorithm that runs well on a CPU, with its deep caches and branch predictors, will most likely perform terribly on a GPU if it involves irregular memory access or divergent control flow. Burtscher and Pingali showed this concretely for Barnes–Hut tree traversal, where thread divergence within warps became the primary bottleneck (Burtscher & Pingali, 2011).

Gravitational N-Body Simulations and Quadratic Scaling

The gravitational N-body problem is central to galactic dynamics: model a galaxy as a collection of particles (stars, dark matter) interacting through gravity, and simulate their evolution over time (Binney & Tremaine, 2008; Zwart et al., 2007). The physics is straightforward. The gravitational force on particle i due to particle j is

$$\boldsymbol{F}_{ij} = G\frac{m_i m_j(\boldsymbol{r}_j - \boldsymbol{r}_i)}{|\boldsymbol{r}_j - \boldsymbol{r}_i|^3} \tag{1}$$

where G is the gravitational constant, m_i and m_j are the particle masses, and $\boldsymbol{r}_i$ and $\boldsymbol{r}_j$ are their positions. The cube of the distance in the denominator combines the inverse-square magnitude with a unit direction vector. Summing over all other particles gives the total force on particle i:

$$\boldsymbol{F}_i = \sum_{j \neq i}^{N} G\frac{m_i m_j(\boldsymbol{r}_j - \boldsymbol{r}_i)}{|\boldsymbol{r}_j - \boldsymbol{r}_i|^3} \tag{2}$$

Every particle interacts with every other particle in the simulation, so the cost scales as $O(N^2)$ per timestep. At $N = 100000$ that is roughly 10^{10} interactions per step, which is feasible on modern GPUs, but far too expensive for the million-step integrations needed to observe phenomena such as spiral arm formation or halo relaxation (Zwart et al., 2007).

Hierarchical Approximation and the Barnes–Hut Algorithm

A variety of approximation techniques have been developed to reduce the computational cost of N-body force evaluation without sacrificing essential physics. The most prevalent "entry level" algorithm is the Barnes–Hut algorithm, which exploits the hierarchical spatial structure of particle distributions (Barnes & Hut, 1986).

Introduced by Barnes and Hut in 1986, this algorithm organises particles into a tree: a quadtree in two dimensions, or an octree in three. During force evaluation, distant groups of particles are replaced by a single effective mass at their centre of mass, collapsing many pairwise interactions into one. An opening-angle parameter θ controls this approximation, trading force accuracy against speed.

The result is a reduction from $O(N^2)$ complexity to roughly $O(N \log N)$. That makes Barnes–Hut a natural fit for large, collisionless systems like galaxies, where global dynamics matter more than exact short-range interactions.

Numerical Integration and Stability Considerations

In scientific galaxy simulations, force evaluation is only half the problem. Accuracy and long term stability also depend on how the equations of motion are integrated. For each particle i, the total gravitational force from all other particles determines its acceleration:

$$\sum_j \boldsymbol{F}_{ij} = m_i \boldsymbol{a}_i \tag{3}$$

Where $\boldsymbol{a}_i$ is the acceleration of the particle i. The acceleration is related to the particle velocity and position through time derivatives,

$$\boldsymbol{a}_i = \frac{d\boldsymbol{v}_i}{dt}, \qquad \boldsymbol{v}_i = \frac{d\boldsymbol{r}_i}{dt} \tag{4}$$

For systems of many interacting particles these equations have no closed-form solution, so their evolution must be approximated numerically. Integration methods advance positions and velocities forward in discrete timesteps of size Δt.

The simplest scheme is the forward Euler integrator (Kreyszig, 2011), which updates the positions of the particles using the current velocity:

$$\boldsymbol{r}_i^{n+1} = \boldsymbol{r}_i^n + \boldsymbol{v}_i^n \Delta t \tag{5}$$

Here the superscript n denotes the discrete timestep, with $\boldsymbol{r}_i^0$ the initial position. Velocities are updated the same way using the current acceleration. Euler is cheap and easy to implement, but it does not conserve energy, as errors accumulate and the simulation drifts into unphysical states over long runs.

Astrophysical N-body codes therefore use symplectic integrators such as Verlet or leapfrog, which preserve the Hamiltonian structure of the equations of motion. These schemes conserve energy far better over long timescales, making them the standard choice for galactic dynamics (Springel, 2005).

In Barnes–Hut simulations, integration error and force-approximation error interact (Salmon & Warren, 1994). For many galactic-scale problems, integration error actually dominates when a poor timestepping scheme is used (Springel, 2005). Choosing the right integrator is just as detrimental as choosing the right force model.

GPU Acceleration of Hierarchical N-Body Methods

The computational demands of N-body simulations make them a natural target for GPUs, with their thousands of small cores executing the same operation across many data elements at once (Fluke et al., 2011). The architecture maps well onto N-body force computation, where each particle's acceleration can be calculated independently (Nyland et al., 2009).

GPU work is expressed as small programs, called shaders, which run on the GPU cores. Shaders were originally designed for rendering: vertex transformations, fragment colouring, texture operations. Early GPU-based scientific computing repurposed these graphics shaders for numerical tasks, encoding computation as rendering passes and storing data in textures (Owens et al., 2008). Languages like CUDA and OpenCL later generalised the model, allowing general-purpose parallel computation unrelated to graphics, including N-body simulations (Fluke et al., 2011; Nyland et al., 2009).

Hierarchical algorithms like Barnes–Hut remain difficult to map onto GPUs, however. Tree construction and traversal introduce irregular memory access and branch divergence, both of which hurt parallel efficiency on SIMD hardware (Burtscher & Pingali, 2011; Karras, 2012). Prior work has addressed this with linearised tree representations, stackless traversal, and space-filling curves to improve memory coherence (Burtscher & Pingali, 2011; Gaburov et al., 2010). These techniques work well on native GPU APIs, but they rely on low-level memory management that has historically been unavailable in browser environments.

Since 1986, many modifications to the Barnes–Hut algorithm have been proposed, such as higher-order multipole expansions, the Fast Multipole Method (FMM), and hybrid approaches (LF & Rokhlin, 2001; Nyland et al., 2009; Wang, 2021). They differ mainly in how distant

interactions are approximated. Barnes–Hut persists because it is simple enough to implement well and accurate enough for most galactic-scale problems.

General Purpose GPU Computation in the Web

Modern web applications increasingly lean on GPU acceleration for interactive, computation-heavy experiences delivered straight through the browser. With GPU access, browsers can run large-model inference, generate embeddings, process multimedia, and render complex visualisations, without requiring users to install any software or manage local hardware (Congote et al., 2011; Sung et al., 2025; Usta, 2024). For students, researchers, and enterprise teams working across devices, browser-based GPU access lowers friction and supports on-demand compute.

WebGL has long been the primary way to access GPU acceleration in the browser. Standardised by the Khronos Group as a JavaScript binding to OpenGL ES, it was designed for real-time graphics rendering across a wide range of devices (Khronos Group, 2024a), and it has enabled a large ecosystem of browser-based visualisation tools and interactive simulations.

The problem is that WebGL's computational model revolves around the graphics pipeline. All computation must be framed as vertex and fragment processing, with data stored in textures and results captured via framebuffers (Congote et al., 2011). General-purpose computation therefore requires working against the API, by reformulating numerical algorithms as sequences of rendering passes, creating non-intuitive data layouts, and requiring multiple shader invocations to emulate iteration, and no direct way to write arbitrary memory locations.

The graphics-centric model works well enough for data-parallel algorithms that map onto image-based representations, but it breaks down for anything requiring irregular memory access, dynamic data structures, or complex control flow (Congote et al., 2011). For Barnes–Hut specifically, WebGL cannot easily express adaptive the fundamentals needed to run it, such as timestepping, hierarchical spatial decomposition, or recursive tree traversal (Burtscher & Pingali, 2011).

For that reason, prior WebGL-based N-body implementations have prioritised visual plausibility over numerical rigour: limited particle counts, softened force models, reduced integration accuracy, or fixed spatial grids instead of adaptive trees (Sengupta et al., 2025). These trade-offs don't matter if the aim of a project is interactive outreach, but they are not adequate for scientific galactic simulations where long-term energy conservation and accurate force evaluation matter.

WebGPU changes this picture, as it is a modern web standard that gives browsers low-level, high-performance access to GPU hardware. Unlike WebGL, it does not force the user to construct programs in terms of a graphics pipeline, but instead through explicit compute shaders, storage buffers, and programmable pipelines (GPU for the Web Working Group, 2026).

The design mirrors native APIs like Vulkan (Khronos Group, 2024b), Metal (Apple Inc., 2024), and Direct3D 12, giving developers fine-grained control over memory layout, synchronisation, and parallel execution (Usta, 2024).

Compute shaders are the central element of WebGPU, as they are general-purpose GPU programs that run independently of the rendering pipeline. Compute shaders express parallel workloads as data processing rather than image generation, which makes them a direct fit for scientific tasks like particle simulation, spatial partitioning, and force evaluation (Sung et al., 2025; Usta, 2024). Algorithms can be structured just as they would be in a native CUDA or Metal implementation, without any graphics-pipeline workarounds (Sengupta et al., 2025).

WebGPU's explicit memory model and flexible buffer abstractions also make it possible to represent complex data structures, such as hierarchical trees and linearised spatial indices directly on the GPU (Sung et al., 2025; Usta, 2024). The data structures are the key to implementing Barnes–Hut in a performant manner, as the algorithm relies on dynamic tree construction and traversal at every timestep, which was entirely impractical under WebGL. With WebGPU, these patterns can run with performance close to native GPU code, subject to browser and hardware constraints (Sengupta et al., 2025).

However, this safety does come at a cost. WebGPU validates every operation and every command buffer submission, and that overhead compounds when many small dispatches are issued. Maczan (Maczan, 2026) characterised this cost for LLM inference across four GPU vendors, three backends, and three browsers, measuring a true per-dispatch overhead of 24–36 µs on Vulkan and 32–71 µs on Metal, with up to 2.2× variation between implementations on the same Metal backend. The takeaway from that work is that per-operation overhead, not kernel compute efficiency, is the bottleneck for dispatch-heavy GPU workloads at small batch sizes. We extend this characterisation from machine-learning inference to scientific simulation, where the dispatch pattern — a fixed pipeline of tree construction and force evaluation passes per timestep — differs fundamentally from the many small operations in a neural-network's forward pass.

2) Methodology

2.1 Research Design and Objectives

We implement and evaluate a gravitational N-body solver with a focus on reproducibility, long-term numerical stability, and computational scalability beyond the $O(N^2)$ cost of direct summation. The solver is written in C++20 using the WebGPU C API and built from a single codebase targeting both native desktop execution (via wgpu-native and Dawn) and browser execution (compiled via Emscripten) (Khronos Group, 2024a). This dual-target approach supports interactive visualisation as well as headless batch runs, so performance and numerical behaviour can be compared under the same conditions across platforms.

On the physics side, we use Newtonian gravity with Plummer softening to avoid the $1/r^2$ singularity (Binney & Tremaine, 2008), and approximate forces with a Barnes–Hut tree to bring the cost down from $O(N^2)$ to roughly $O(N \log N)$ (Barnes & Hut, 1986). The tree itself is a Linear Bounding Volume Hierarchy (LBVH), built and traversed entirely on the GPU each timestep using Karras's parallel construction method (Karras, 2012). We integrate with a second-order symplectic leapfrog (kick–drift–kick), which conserves energy far better than forward Euler over long integrations (Springel, 2005). All of this runs on WebGPU, which gives us general-purpose compute shaders through the same API on both native and browser targets.

The evaluation is structured around three main research questions:

1. **Scalability**: How does runtime per timestep scale with N for a WebGPU Barnes–Hut implementation, and where are the bottlenecks within the GPU pipeline?
2. **Abstraction overhead**: What performance cost does the WebGPU abstraction layer impose relative to Apple's native Metal, and how does this vary across WebGPU implementations (wgpu-native, Dawn, browser)?
3. **Browser feasibility**: Can the same codebase, compiled to WebAssembly and running in a browser, achieve acceptable throughput and numerical consistency compared to native execution?

As discussed in the literature review, WebGPU is the only current GPU API that combines compute shaders, storage buffers, and atomic operations with browser deployability, which are the primitives our tree-based solver needs, available on both native and web targets from a single codebase. Table 1 summarises how it compares to the alternatives. The relevant constraints for our implementation are: all GPU arithmetic is 32-bit (no `f64`), which caps numerical precision; dispatch scheduling overhead can dominate at small N; and buffer behaviour varies across backends and devices (Sengupta et al., 2025). These constraints are tested directly through the three research questions above.

Criterion	CUDA	OpenCL	WebGL	WebGPU
Compute shaders	Yes	Yes	No	Yes
Storage buffers	Yes	Yes	No	Yes
Atomic operations	Yes	Yes	No	Yes
Browser deployment	No	No	Yes	Yes
Cross-vendor GPU	No	Yes	Yes	Yes
Native deployment	Yes	Yes	No	Yes
64-bit GPU float	Yes	Varies	No	No

Table 1: Comparison of GPU compute APIs against criteria required for hierarchical N-body simulation. WebGPU is the only API that combines compute shader support with browser deployability.

2.2 Physical Model and State Representation

In our simulation, each particle represents a mass element evolving under self-gravity in an isolated (open) domain. We adopt a natural unit system in which the gravitational constant $G = 1$, the unit of mass M_0 is defined by the total system mass, and the unit of length r_0 is set by the characteristic scale of the initial conditions (e.g., the Plummer scale length a in Scenario B). Time is measured in units of $t_0 = \sqrt{r_0^3/(GM_0)}$, the free-fall timescale of the system. All the quantities reported in subsequent sections (energies, momenta, and timescales) are expressed in terms of these natural units unless stated otherwise. The softened acceleration of particle i due to all other particles is

$$\boldsymbol{a}_i = G \sum_{j \neq i} m_j \frac{\boldsymbol{r}_j - \boldsymbol{r}_i}{\left(\|\boldsymbol{r}_j - \boldsymbol{r}_i\|^2 + \varepsilon^2\right)^{\frac{3}{2}}} \tag{6}$$

where $\boldsymbol{r}_i$ and $\boldsymbol{r}_j$ are the position vectors of particles i and j, m_j is the mass of particle j, and ε is a softening length (default 0.5, configurable). The softening parameter introduces a Plummer-type potential that suppresses the $1/r^2$ singularity at short range, preventing erratic and divergent accelerations during close interactions (Binney & Tremaine, 2008). This formulation is equivalent to treating each particle as a smooth mass distribution of characteristic radius ε rather than a single point mass.

On the GPU, a particle's state is stored as packed four-component vectors (`vec4<f32>`), with mass in the w component of the position vector: (x, y, z, m). This packing cuts the number of buffer reads during force evaluation compared to separate position and mass buffers. We considered a Structure-of-Arrays (SoA) layout but rejected it after further analysis, since the BVH force shader (95–99% of step time) never reads particle masses directly, as it uses the aggregated node masses from the bottom-up pass. The shaders that do read particle mass (leaf

initialisation, drift) always need to access the position and mass together, so separating them would hurt cache performance. The shaders that only read positions (bounding box, Morton code) account for less than 1% of step time, making any bandwidth saving negligible. Table 2 summarises the storage layout for all three state arrays.

Component	`.x`	`.y`	`.z`	`.w`
`positions[i]`	x_i	y_i	z_i	m_i
`velocities[i]`	$v_{x,i}$	$v_{y,i}$	$v_{z,i}$	0
`accelerations[i]`	$a_{x,i}$	$a_{y,i}$	$a_{z,i}$	0

Table 2: Particle state layout using `vec4<f32>` packing. Mass is stored in the w component of the position vector to reduce buffer count and memory bandwidth during force evaluation.

All GPU kernels run in 32-bit floating point, which maximises throughput and matches current WebGPU capabilities. Diagnostic quantities such as the total energy and linear momentum, are computed on the CPU in double precision (64-bit) to reduce accumulation error over long integrations. This split is a practical concession, since WebGPU does not support 64-bit GPU arithmetic (Sengupta et al., 2025).

2.3 Time Integration

In order to represent time in a discrete manner, we integrate with a fixed-timestep, second-order symplectic leapfrog in kick–drift–kick (KDK) form (Verlet, 1967). In the following equations, the superscript n denotes the timestep index, and $\boldsymbol{r}_i^n$, $\boldsymbol{v}_i^n$, $\boldsymbol{a}_i^n$ represent the position, velocity, and acceleration of particle i at step n. With timestep Δt (default 10^{-3}), the update has three stages. A half-kick advances velocities by half a step using the current accelerations:

$$\boldsymbol{v}_i^{n+1/2} = \boldsymbol{v}_i^n + \frac{\Delta t}{2}\boldsymbol{a}_i^n \tag{7}$$

Next, a drift advances positions using the half-stepped velocities:

$$\boldsymbol{r}_i^{n+1} = \boldsymbol{r}_i^n + \Delta t \boldsymbol{v}_i^{n+1/2} \tag{8}$$

Accelerations $\boldsymbol{a}_i^{n+1}$ are then recomputed from the updated positions using the hierarchical force evaluation described in the following section. Finally, a second half-kick completes the velocity update:

$$\boldsymbol{v}_i^{n+1} = \boldsymbol{v}_i^{n+1/2} + \frac{\Delta t}{2}\boldsymbol{a}_i^{n+1} \tag{9}$$

We chose leapfrog for its well-known long-term stability in gravitational dynamics (Springel, 2005). Unlike forward Euler, which introduces secular energy drift proportional to Δt, the leapfrog is time-reversible and symplectic: it preserves phase-space volume, producing

bounded energy oscillations rather than monotonic growth (Binney & Tremaine, 2008). For long integrations, this matters far more than raw speed.

The most expensive step in each integration cycle is the evaluation of accelerations $\boldsymbol{a}_i$, which is addressed through hierarchical approximation in the following section.

2.4 Hierarchical Force Evaluation

Rather than computing all $N(N-1)/2$ pairwise interactions directly, our solver groups distant particles into tree nodes and approximates each group as a single equivalent mass.

Each internal node of the tree stores a total mass $M = \sum_{i\in \text{ node}} m_i$ and a centre of mass $\boldsymbol{R} = \left(\sum_{i\in \text{ node}} m_i \boldsymbol{r}_i\right)/M$, computed by aggregating over all particles in that node's subtree. When a node is accepted as a monopole, its gravitational contribution to particle i is

$$\boldsymbol{a}_{i,\text{node}} = GM \frac{\boldsymbol{R} - \boldsymbol{r}_i}{(\|\boldsymbol{R} - \boldsymbol{r}_i\|^2 + \varepsilon^2)^{\frac{3}{2}}} \tag{10}$$

The classical Barnes–Hut algorithm uses an octree, where space is recursively divided into eight equal sub-cubes, producing a tree whose structure is determined by the spatial partition rather than the particles themselves. On a GPU, octrees are awkward, since each node has up to eight children, many of which may be empty, and the recursive top-down construction is inherently serial. A Bounding Volume Hierarchy (BVH) avoids both problems. A BVH is a binary tree in which each internal node stores an axis-aligned bounding box (AABB) that tightly encloses the particles in its subtree. Because the tree is binary (two children per node, not eight), traversal requires fewer branch decisions per level, and the total node count is fixed at $2N-1$ regardless of the spatial distribution. Most importantly, a BVH can be built bottom-up from a sorted particle list in a fully parallel, non-recursive pipeline, which is the Linear BVH (LBVH) construction described below. The trade-off is that BVH bounding boxes can overlap (unlike octree cells, which partition space without gaps), so the opening criterion must use the actual node extent rather than a uniform cell width.

We implement the tree as a BVH constructed and traversed entirely on the GPU each timestep.

An internal node is accepted as a monopole when it is sufficiently small relative to its distance from the target particle. The opening criterion tests whether the squared maximum extent of the node's axis-aligned bounding box (AABB) satisfies $\text{maxExtent}^2/d^2 < \theta^2$, where $\text{maxExtent} = \max(\Delta x, \Delta y, \Delta z)$ and $d^2 = \|\boldsymbol{r}_i - \boldsymbol{R}\|^2$. This extent-based criterion reflects the non-cubic node shapes that arise in a BVH more accurately than a uniform half-width, and is expressed in squared form to avoid a per-node square root. Salmon and Warren provide rigorous

error bounds for these opening criteria, showing that the conventional Barnes–Hut criterion can admit unbounded errors for $\theta \geq 1/3$ in pathological configurations (Salmon & Warren, 1994).

Figure 1 illustrates the geometry of both criteria. The default opening angle $\theta = 0.75$ is used as a practical balance between accuracy and performance. For comparison, GADGET-2 uses θ values in the range 0.5 to 0.7 for cosmological simulations where higher force accuracy is required (Springel, 2005), while the original Barnes and Hut paper used $\theta = 1.0$ (Barnes & Hut, 1986). The parameter sweeps in the evaluation protocol (see Section 2.10) systematically characterize the accuracy–performance trade-off across $\theta \in \{0.3, 0.5, 0.7, 1.0\}$.

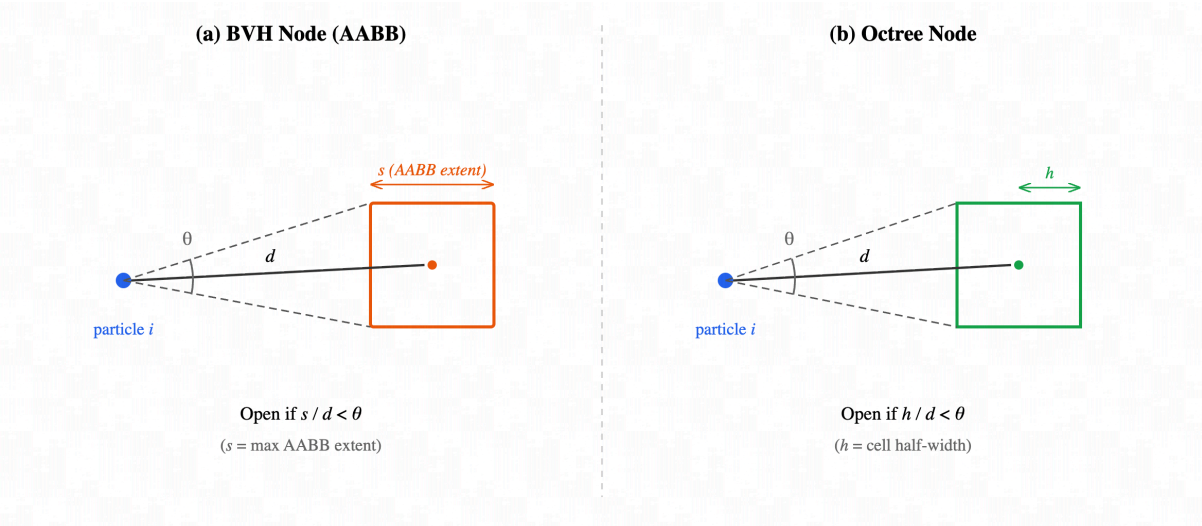


Figure 1: Geometry of the opening criterion. A node is approximated as a monopole when its angular size, as measured by extent/d, falls below the threshold θ. Left: BVH variant using maximum AABB extent (this work). Right: classical octree variant using cell half-width, shown for comparison with the literature.

2.5 Software Architecture and Execution Modes

The implementation is written in C++20 for host-side orchestration and WGSL for GPU compute and rendering shaders, using the WebGPU C API directly without wrapper libraries. CMake handles the build with pinned dependency versions for deterministic builds. Three backends are supported: wgpu-native (gfx-rs contributors, 2024) (Rust-based), Dawn (Google, 2024) (Google's implementation), and Emscripten (cross-compiling to WebAssembly for browser deployment).

All physics, such as the integration, tree construction, and force evaluation, runs on the GPU every step. The CPU handles only diagnostics: at configurable intervals, positions and velocities are read back through staging buffers and energy and momentum are computed in double precision. No per-step CPU physics overhead is incurred in the primary execution path.

2.6 WebGPU Compute Methodology

Simulation state is stored in WebGPU storage buffers using the packed `vec4<f32>` layout described in Table 2: positions (with mass in the w component), velocities, and accelerations. The BVH is stored as a flat array of $2N - 1$ nodes, each containing a centre of mass, total mass, and axis-aligned bounding box. A uniform parameters buffer shares simulation parameters between host and shader code. In-place updates are used throughout: there is no double buffering of positions, velocities, or accelerations. Correct ordering between compute passes relies on WebGPU's implicit storage-buffer synchronisation between dispatches within a single command buffer submission. Bind groups are cached and reused across frames to avoid per-frame recreation overhead.

The WGSL compute shaders fall into two groups: integration and force evaluation (direct-summation baseline and BVH traversal), and LBVH construction in six passes (Karras, 2012; Morton, 1966). Workgroup sizes are tuned per kernel (128 for force evaluation, 256 for integration and tree building) with the rationale discussed in the optimisation section below.

2.6.1 Per-timestep execution sequence

Each timestep is recorded into a single command encoder and submitted as one command buffer. The six sequential passes, illustrated in Figure 2, are: (1) a half-kick that advances velocities by $\Delta t/2$ using the current accelerations; (2) a drift that advances positions by Δt using the half-stepped velocities; (3) a full LBVH rebuild comprising six sub-passes (global AABB reduction, Morton code generation, radix sort, Karras topology construction, leaf initialisation, and bottom-up aggregation); (4) BVH force evaluation, in which each particle traverses the tree iteratively using a fixed-depth explicit stack (depth 64, sufficient for all tested N); (5) a second half-kick that completes the velocity update; and (6) an optional diagnostics readback at configurable intervals, in which positions and velocities are copied to the CPU via staging buffers for double-precision energy and momentum computation.

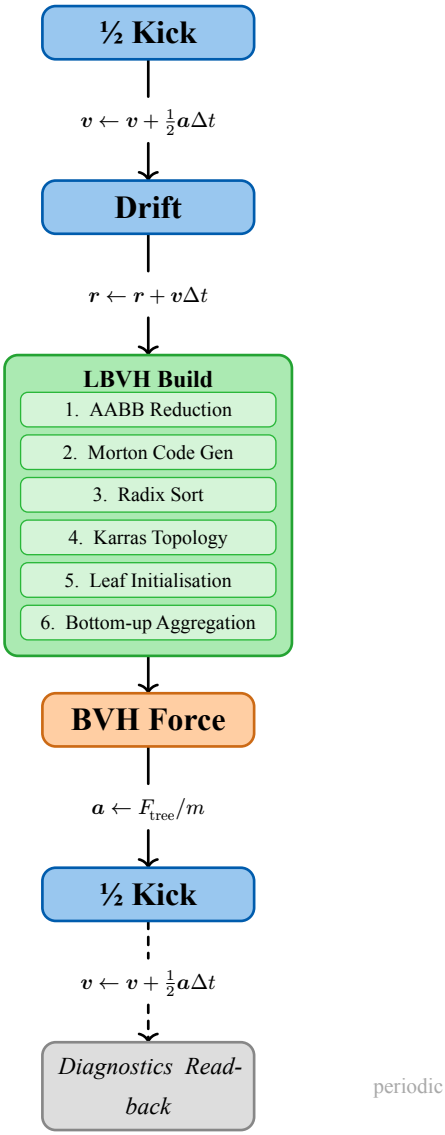


Figure 2: Per-timestep GPU execution pipeline for the leapfrog integrator. All passes are recorded into a single command buffer. The LBVH build comprises six sub-passes with implicit barrier synchronisation between each. Diagnostics readback occurs at configurable intervals.

2.6.2 GPU tree traversal

Because GPU compute shaders do not support recursion since they don't have a call stack, tree traversal is implemented iteratively in the BVH force shader. One GPU thread is assigned per particle, and the thread maintains an explicit stack of node indices (maximum depth 64), beginning from the root. At each step, a node is either accepted as a monopole (if it is a leaf, or if the opening criterion is satisfied) and its gravitational contribution is accumulated, or it is expanded by pushing its two children onto the stack. This explicit-stack approach preserves the Barnes–Hut approximation structure while accommodating the lack of a call stack on GPU hardware and limiting branch divergence across threads within a workgroup (Burtscher & Pingali, 2011; Karras, 2012; Nyland et al., 2009).

2.6.3 GPU LBVH construction

The Linear Bounding Volume Hierarchy is built fully on the GPU each timestep, following the parallel construction method of Karras (Karras, 2012). The construction proceeds through six compute dispatches, illustrated in Figure 4:

1. **Global bounding box reduction.** A two-pass parallel reduction over all particle positions computes the axis-aligned bounding box of the entire system. This bounding box defines the coordinate range used to normalise positions in the next step.
2. **Morton code generation.** Each particle's position is normalised to a $[0, 1023]^3$ integer grid within the global bounding box, and the three 10-bit integer coordinates are interleaved to produce a single 30-bit Morton code (Morton, 1966). The Morton code maps a three-dimensional position to a one-dimensional index along a Z-order (Morton) space-filling curve, so that particles that are spatially close in 3D tend to receive numerically similar codes. Figure 3 illustrates this principle in two dimensions: the Z-curve (red dashed line) visits grid cells in an order that preserves spatial locality, so that after sorting by Morton code, particles in the same cell are stored contiguously in the sorted array. The same principle extends to three dimensions with octant interleaving.

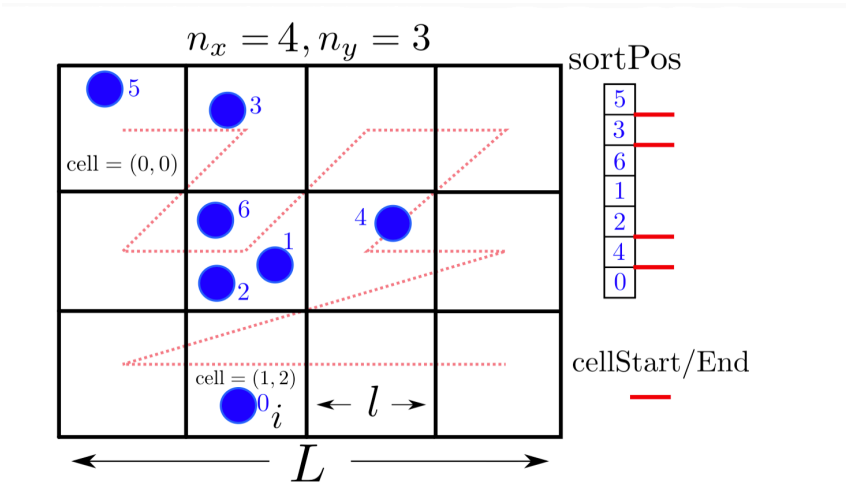


Figure 3: Spatial binning via a Z-order (Morton) space-filling curve in 2D. The dotted red line traces the curve through the grid; after sorting by Morton code, spatially adjacent particles are stored contiguously in the sorted array (right). Adapted from Peláez (Peláez, 2022).

1. **Radix sort.** The Morton codes and their associated particle indices are sorted into ascending order using a parallel radix sort. The radix sort processes the 32-bit keys in four passes (8 bits per pass), performing a prefix-sum histogram within each pass to determine output positions. This approach was chosen over the bitonic sort network (Batcher, 1968) used in an earlier version of the implementation, because radix sort achieves $O(N)$ work complexity for fixed-width keys and scales more predictably on GPU hardware. An alternative is the onesweep

radix sort, which is the most performant variant on native GPU APIs, but its reliance on fine-grained device-scope atomic operations makes it difficult to implement efficiently in WebGPU, where atomics are limited to workgroup and storage-buffer scope.

2. **Karras topology construction.** The sorted Morton codes define a binary radix tree whose internal structure is determined entirely by the codes themselves. For each internal node i, the Karras algorithm computes a direction and range by examining the *delta function* $\delta(i, j)$, defined as the number of leading zero bits in the bitwise XOR of Morton codes k_i and k_j. Two codes that share a long common prefix (high δ) correspond to particles that are spatially close, because their Morton codes agree on the most significant bits of their interleaved coordinates. The algorithm determines each internal node's children by finding the split position within its range where the common prefix length changes. Duplicate Morton codes (which arise when two particles fall in the same grid cell) are handled by appending the particle index as a tie-breaker to ensure a unique ordering (Karras, 2012).
3. **Leaf initialisation.** Each leaf node is assigned the position, mass, and a point AABB corresponding to its sorted particle.
4. **Bottom-up aggregation.** Internal-node bounding boxes and centres of mass are computed in a bottom-up pass. An atomic counter per internal node tracks how many of its two children have been processed; the second thread to arrive at a node computes the merged AABB and mass-weighted centre of mass from both children, ensuring correctness without explicit synchronisation barriers.

The resulting BVH is immediately traversable without any CPU-side construction or data upload, eliminating what would otherwise be a per-step CPU–GPU transfer bottleneck.

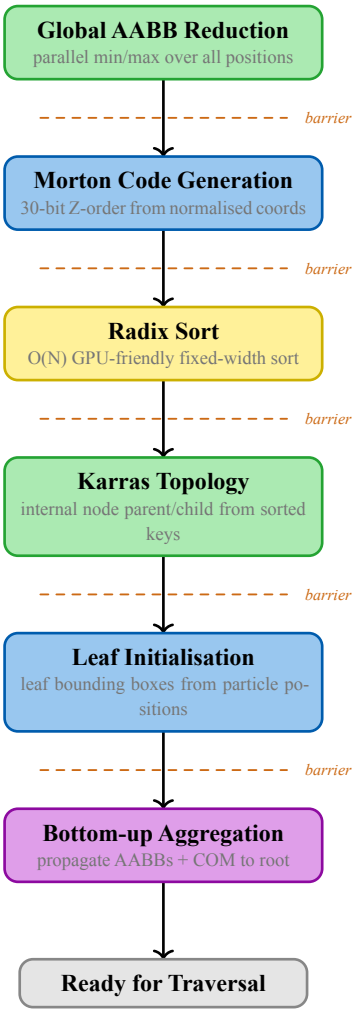


Figure 4: LBVH construction pipeline. Six compute dispatches build the tree entirely on-device. Dashed lines represent implicit storage-buffer barriers between passes. The atomic-counter aggregation in pass 6 ensures correct bottom-up propagation of bounding boxes and centres of mass.

2.7 Force Traversal Optimisation

Profiling showed that BVH force evaluation accounts for 94–99% of total step time at all tested N, with LBVH construction contributing less than 1% even at $N = 100000$. We therefore focused all optimisation effort on the traversal shader, applying four changes and benchmarking each independently before combining them.

The baseline traversal shader performs three operations per node visit: a full 64-byte struct read from global memory, an opening-criterion computation involving approximately 20 floating-point operations including transcendental functions, and a stack-based depth-first traversal with no spatial ordering. Each represents a different bottleneck class (memory bandwidth, ALU throughput, and cache efficiency respectively), and the optimisations target all three.

Precomputed opening radius. The opening criterion depends only on node-intrinsic properties that are invariant across particle interactions. In the baseline, these are recomputed for every particle–node pair. The optimised version precomputes a scalar opening radius per node during the bottom-up aggregation pass, computed as $\mathrm{halfExtent} = \|\boldsymbol{R} - \boldsymbol{c}\| \cdot (1 + 0.6 \log_2(\max(M, 1)))$ where $\boldsymbol{R}$ is the centre of mass, $\boldsymbol{c}$ is the nearest AABB corner, and M is the node's total mass. The mass-adaptive scaling widens the opening radius for massive nodes, making the traversal more conservative (and more accurate) where gravitational influence is strongest. The precomputed radius is stored in an otherwise unused field, and the per-interaction test reduces to a single distance comparison $d^2 > r^2$. This eliminates redundant computation across all particle interactions, giving us a $1.74\times$ speedup at $N = 100000$.

Compact traversal nodes. The force shader reads a 64-byte BVH node per visit but uses only 28 bytes (centre of mass, opening radius, child indices). A compaction pass copies these fields into a 32-byte traversal buffer, reducing memory bandwidth per node visit. This provides an additional 3% improvement at large N.

Workgroup size tuning. Testing workgroup sizes of 64, 128, and 256 with otherwise identical code showed that 128 threads per workgroup (four SIMD groups on the Apple M2) provided the best balance between register pressure and occupancy for N up to 50,000. This parameter is hardware-dependent and would require re-tuning on other GPUs.

Morton-ordered particle access. In the baseline, GPU thread i processes particle i, so adjacent threads may handle spatially distant particles that traverse different parts of the tree, causing poor cache utilisation. The optimised version uses the Morton-code sort order already computed during LBVH construction: thread i processes the particle at sorted index i, so adjacent threads handle spatially nearby particles that visit similar tree paths. This improves L1/L2 cache hit rates during traversal, yielding an additional 20% speedup at $N = 100000$.

We also tried two approaches that did not work. Near-far child ordering (pushing the farther child first so the nearer subtree is processed first) caused cache thrashing from the extra node reads, making force computation 10–30% slower. Warp-coherent traversal using subgroup operations was not feasible because WGSL subgroup support is limited to experimental extensions on Dawn.

N	Baseline (ms)	Optimised (ms)	Speedup
1,000	5.8	6.9	0.84×
5,000	9.9	8.0	1.24×
10,000	11.3	11.5	∼1.0×
50,000	109.5	67.2	1.63×
100,000	396.2	183.9	2.15×

Table 3: Combined effect of all four traversal optimisations. The overhead of the compaction pass and sort-index lookup causes a small regression at $N = 1000$; the crossover where optimisations break even is approximately $N = 3000$.

2.8 Rendering and Interactive Operation

In interactive mode, particles are rendered as instanced billboard quads with additive blending. The vertex shader reads positions directly from the physics storage buffers, avoiding per-frame data uploads. A toggleable trail effect improves visibility in sparse scenarios (e.g. the two-body orbit): a screen-space accumulation buffer reprojects the previous frame's trails into the current camera view, fades them by a configurable amount, and composites new particle positions on top with additive blending, using a ping-pong texture pair. An ImGui overlay (Figure 5) provides interactive control of simulation parameters and displays real-time diagnostics. In headless mode, rendering is skipped entirely for pure throughput measurement.

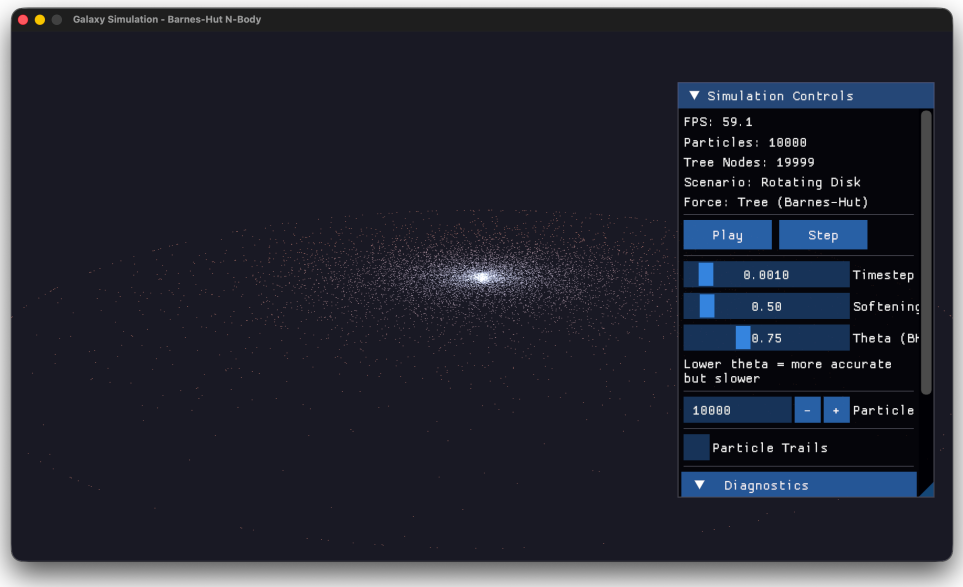


Figure 5: Interactive mode showing the ImGui control overlay. Simulation parameters, timing diagnostics, and rendering options (including the particle trail effect) can be adjusted at runtime.

2.9 Initial Conditions and Benchmark Scenarios

All experiments start from synthetic initial conditions and produce derived outputs (trajectories, diagnostic scalars, timing logs), enabling controlled and repeatable comparisons across parameter sweeps. Each experiment is fully specified by its simulation parameters: scenario type, seed, N, Δt, θ, softening ε, and step count. Three benchmark scenarios are defined, with initial particle distributions shown in Figure 6 and complete parameter specifications for all experiment groups listed in Table 15.

2.9.1 Scenario A: two-body circular orbit

Scenario A places two equal-mass particles ($m = 1000$ each, $N = 2$) separated by $d = 10$ units along the x-axis, with tangential velocities along the z-axis computed for a softened circular orbit:

$$v = \sqrt{\frac{Gmd^2}{2(d^2 + \varepsilon^2)^{\frac{3}{2}}}} \tag{11}$$

This is the simplest possible validation of integrator correctness. With the right timestep and softening, the two particles should maintain a stable circular orbit indefinitely under leapfrog. We validate quantitatively, not visually: the energy drift $\Delta E / \ |E(0)|$ over the full integration is measured directly, and any departures from circularity can be isolated without confounding effects from hierarchical force approximation at a large N.

2.9.2 Scenario B: Plummer sphere

A Plummer sphere is a spherically symmetric, self-gravitating stellar system with a density that falls off smoothly with distance from the centre. It was first introduced by Plummer (Plummer, 1911) as an empirical fit to the light profiles of globular clusters, and its density profile is given by $\rho(r) \propto \left(1 + r^2/a^2\right)^{-5/2}$, where a is a scale length that sets the size of the core. Because the Plummer model has known analytic properties, such as closed-form expressions for the potential, escape velocity, and distribution function, it is widely used as a standard test case for N-body codes (Aarseth et al., 1974; Binney & Tremaine, 2008). Deviations from the expected equilibrium behaviour provide a direct diagnostic of force accuracy and integration stability.

Scenario B generates a Plummer sphere with $N \in [10^3, 10^5]$ (depending on hardware) and scale length $a = 5$. Particle radii are sampled via the inverse cumulative distribution function

$$r = \frac{a}{\sqrt{u^{-2/3} - 1}} \tag{12}$$

with u clamped to $[0.001, 0.999]$. Angular coordinates are isotropic: $\cos(\theta)$ is drawn uniformly and the azimuthal angle ϕ is drawn uniformly on $[0, 2\pi)$. Particle speeds are sampled via rejection sampling using

$$g(q) = q^2(1-q^2)^{\frac{7}{2}} \tag{13}$$

against the local escape velocity

$$v_e = \sqrt{\frac{2GM}{\sqrt{r^2+a^2}}} \tag{14}$$

with isotropic velocity directions. This scenario tests tree accuracy and long-term stability in a compact three-dimensional distribution. The Plummer softening parameter ε in the force model is conceptually related to the Plummer scale length a in the initial conditions, though the two are independently configurable. The key sweep variables are θ, ε, and Δt; the primary limitation is that the spherical geometry does not emphasize disk morphology such as spirals and bars.

2.9.3 Scenario C: rotating exponential disk (galaxy-like morphology test)

Scenario C generates a rotating disk with $N \in [10^4, 10^5]$ to evaluate long-term evolution and visually interpretable galactic dynamics. The surface density profile follows an exponential distribution (Freeman, 1970), with particle radii drawn from an exponential distribution (rate parameter 0.08) and clamped to a maximum of 50 units, and azimuthal angles drawn uniformly. Vertical height is drawn from a normal distribution $\mathcal{N}(0, 0.3)$ scaled by $\frac{1}{1+0.5r}$ to produce a thinner disk at larger radii. Particle masses are drawn uniformly from $[0.5, 2.0]$.

Circular velocities are assigned using an approximate enclosed-mass estimate. For particles with $r > 0.1$, the tangential velocity is

$$v = 0.5\sqrt{\frac{M_{\text{enclosed}}}{r}} \tag{15}$$

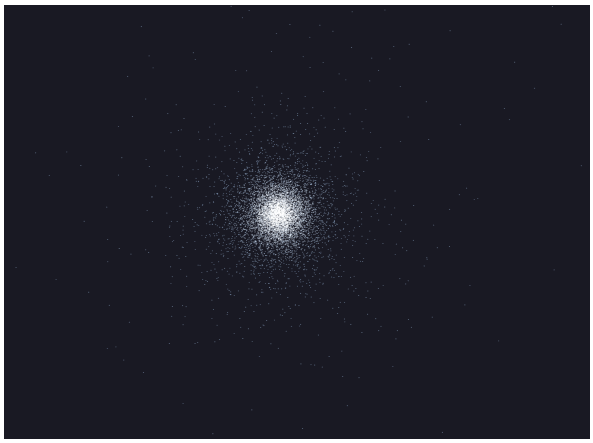
with the velocity directed tangentially. This simplified dynamical setup is not a full multi-component Milky Way model: the enclosed-mass estimate is approximate, and no bulge or halo component is included. Nevertheless, the disk geometry provides a morphologically rich test case in which the formation and persistence of spiral structure, bars, and other large-scale features serve as qualitative validation of the solver's long-term behavior. The key sweep variables are disk scale length, thickness, velocity dispersion, θ, and Δt.

2.9.4 Sampling and robustness across seeds

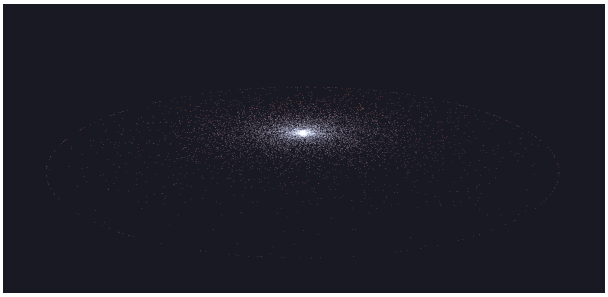
Because the initial conditions are stochastic, we assess robustness by repeating runs with different random seeds and comparing diagnostics and timing. A run is valid if it completes without NaN or overflow and produces consistent parameter logs. We deliberately retain unsta-

ble configurations (such as excessively large Δt) as documented failures rather than silently excluding them.

(a) Two-body orbit with particle trails



(b) Plummer sphere



(c) Exponential disk

Figure 6: Initial particle distributions for the three benchmark scenarios. (a) two-body orbit. (b) Plummer sphere with $N = 10000$. (c) exponential disk with $N = 50000$.

2.10 Evaluation Protocol

The evaluation compares the WebGPU solver against a native Metal baseline and across multiple WebGPU implementations, using a consistent benchmarking protocol throughout.

The primary baseline is UniSim (Gomes, 2024), an open-source Barnes–Hut N-body solver written directly against the Metal API. UniSim runs on the same Apple M2 GPU with the same Metal driver as all four WebGPU implementations, so the performance difference isolates what the WebGPU abstraction layer costs. A direct $O(N^2)$ summation path within our solver serves as a secondary baseline for finding the crossover at which hierarchical force evaluation pays off.

The primary metric is runtime per timestep (ms/step), decomposed into three components: tree build time (GPU LBVH construction), force evaluation time (BVH traversal), and integration time (kick and drift dispatches). This decomposition reveals which phase dominates at each particle count. For the cross-backend comparison, we quantify abstraction overhead as the ratio of WebGPU ms/step to native Metal ms/step at matched N and parameters. For the browser

comparison, subtracting native GPU time from browser wall-clock time isolates the fixed per-step scheduling overhead.

Energy drift ($\Delta E(t) = |E(t) - E(0)| \ / \ |E(0)|$) is reported as a secondary observation for $N \leq 5000$, where potential energy can be computed via direct pair summation. Momentum conservation is monitored throughout. These metrics characterise the 32-bit precision floor of WebGPU rather than constituting a primary research question.

Accurate GPU timing requires the CPU to wait for GPU work to complete before reading the clock. The synchronisation mechanism differs across backends: wgpu-native provides a blocking device poll, while Dawn and Emscripten use a buffer-map fence (a small staging buffer whose map callback fires only after all prior GPU work completes). Two timing modes are used: whole-step timing, in which a single command encoder records the entire timestep and the GPU is synchronised once at the end (used for total ms/step comparisons), and per-phase timing, in which separate command encoders and GPU synchronisations are issued per phase (used for the LBVH pass breakdown, at the cost of additional synchronisation overhead).

All measurements follow the benchmarking protocol described in the experiments section: 50 warmup steps discarded, 100 measured steps, with mean $\pm$ standard deviation, 95% confidence interval, and coefficient of variation reported for each configuration (Maczan, 2026). All initial conditions are generated deterministically (default seed 42), and the execution environment is recorded alongside each run.

3) Experiments

This section specifies the hardware environment and experiment configurations used to evaluate the solver against the three research questions. All experiments run in headless batch mode, eliminating rendering overhead and producing deterministic CSV logs for post-processing.

3.1 Experimental Platform

All experiments were executed on a single workstation whose configuration is summarised in Table 4. The GPU adapter name and WebGPU backend are reported by the application at startup and recorded alongside each run.

Component	**Specification**
GPU / CPU	Apple M2 (8-core GPU, 4P+4E CPU, unified memory)
RAM	16 GB unified memory
Operating System	macOS 26.2 (Darwin 25.2.0)
Native WebGPU	wgpu-native (Rust-based, Metal backend) via WebGPU-distribution v0.2.0
Native WebGPU (alt.)	Dawn (Google, Metal backend) via WebGPU-distribution v0.2.0
Browser (Chromium)	Chrome 146.0.7680.178 (arm64)
Browser (WebKit)	Safari 26.2
Metal baseline	UniSim (Gomes, 2024) (native Metal Barnes–Hut)
Compiler	Apple Clang 17.0.0, C++20, Release (`-O3 -DNDEBUG`)
Timing	CPU wall-clock via `std::chrono::high_resolution_clock` (sub-µs resolution), bracketing backend-specific GPU fences: `wgpuQueueOnSubmittedWorkDone` + poll (wgpu-native), buffer-map fence (Dawn / Emscripten). `--sync-timing` for total ms/step; `--benchmark-passes` for per-component breakdown.

Table 4: Hardware and software configuration. All experiments run on the same Apple M2 system. The four WebGPU implementations and the native Metal baseline all use the same Metal graphics driver, isolating the overhead of each abstraction layer.

Each run is executed in headless batch mode, with all simulation parameters (scenario type, N, Δt, θ, softening ε, integrator, tree type, force method, step count, and random seed) specified at invocation. The CSV output records twelve columns per step: step number, simulation time, kinetic energy, potential energy (zero when $N > 5000$), total energy, energy drift, three momentum components (p_x, p_y, p_z), and three timing components (tree build, force evaluation, and integration in milliseconds).

3.2 Benchmarking Protocol

We follow established practices for GPU benchmarking (Maczan, 2026). Each configuration is warmed up by discarding the first 50 steps, during which pipeline compilation, buffer allocation, and caching stabilise. Timing is then collected over the next 100 steps and reported as mean $\pm$ standard deviation, with the 95% confidence interval via the t-distribution. The coefficient of variation ($\mathrm{CV} = \sigma/\mu$) quantifies run-to-run stability; configurations with $\mathrm{CV} > 10\%$ are flagged and investigated. For the cross-backend comparison (Group 6), positions are held frozen — forces are computed but not applied — to isolate dispatch and scheduling overhead from physics-dependent variation in tree structure. This frozen-state protocol ensures all backends execute identical GPU workloads, so measured differences reflect only the WebGPU implementation overhead and host-side scheduling.

3.3 Experiment Groups

The experiments are organised into seven groups, each targeting one or more research questions or characterising numerical quality (Table 5). Exact command-line invocations for reproducing each group are provided in the Appendix.

Group	Scenario	Variable	Runs	RQ
1: Two-body validation	A (two-body)	Δt	5	Qual.
2a: N-scaling	B (Plummer)	$N \in \{100\text{–}10^5\}$	9	RQ1, RQ3
2b: Theta sweep	B (Plummer)	$\theta \in \{0.3, 0.5, 0.7, 1.0\}$	4	Qual.
2c: Timestep sweep	B (Plummer)	$\Delta t \in \{5 \times 10^{-5}\text{–}5 \times 10^{-3}\}$	5	Qual.
2e: Softening sweep	B (Plummer)	$\varepsilon \in \{0.1\text{–}2.0\}$	5	Qual.
3a: Disk N-scaling	C (disk)	$N \in \{10^4\text{–}10^5\}$	5	RQ1, RQ3
4: Direct vs tree crossover	B (Plummer)	force method $\times$ N	12	RQ1
5: Native vs browser	B (Plummer)	execution platform $\times$ N	9	RQ3
6: Cross-backend comparison	B (Plummer)	WebGPU impl $\times$ N	12	RQ2, RQ3
7: LBVH pass breakdown	B (Plummer)	per-pass timing $\times$ N	5	RQ1

Table 5: Summary of experiment groups, swept variables, and targets. Qual. denotes numerical quality characterisation, reported as a secondary observation rather than a primary research question.

3.3.1 Group 1: Two-Body Orbit Validation (Scenario A)

This group verifies integrator correctness using the two-body orbit configuration ($N = 2, m = 1000$ each). The timestep Δt is swept over $\{0.0001, 0.0005, 0.001, 0.005, 0.01\}$ using the leapfrog integrator, yielding five runs of 50,000 steps each. All runs use the GPU BVH tree

path with $\theta = 0.75$, $\varepsilon = 0.5$, and seed 42. The primary diagnostic is the energy drift $\frac{\Delta E(t)}{|E(0)|}$ over the full integration, with secondary attention to momentum magnitude and orbit stability.

3.3.2 Group 2: Plummer Sphere Parameter Sweeps (Scenario B)

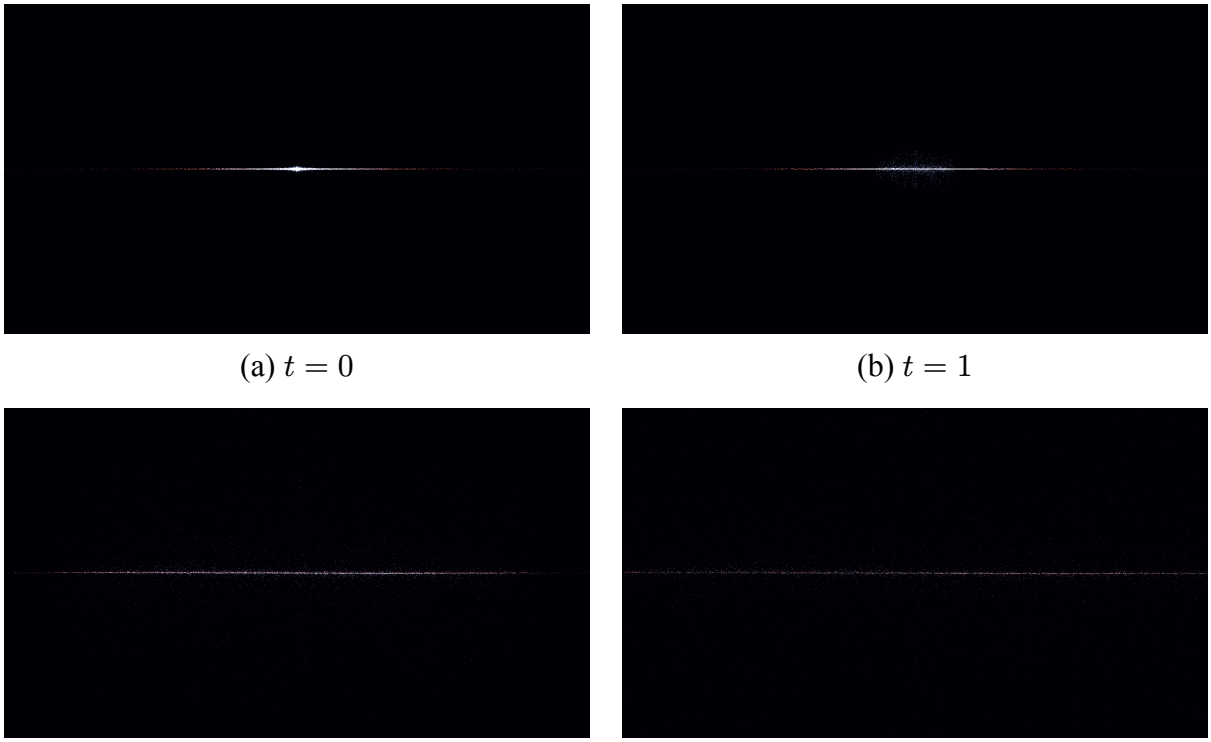
The Plummer sphere is the primary quantitative testbed because of its symmetric initial conditions and known analytic equilibrium properties. Four sub-groups isolate individual parameter effects.

Sub-group 2a (N-scaling) varies N over $\{100, 500, 1000, 2000, 5000, 10000, 25000, 50000, 100000\}$ with all other parameters fixed at defaults ($\Delta t = 0.001$, $\theta = 0.75$, $\varepsilon = 0.5$, leapfrog, GPU LBVH). Each run executes 1,000 steps. This sub-group directly addresses RQ1 (scalability) and RQ3 (feasibility at large N).

Sub-groups 2b, 2c, and 2e isolate the effects of individual parameters at fixed $N = 5000$ (chosen because potential energy is computed at this size, enabling full energy tracking). Sub-group 2b sweeps $\theta \in \{0.3, 0.5, 0.7, 1.0\}$; sub-group 2c sweeps $\Delta t \in \{5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}, 5 \times 10^{-3}\}$; and sub-group 2e sweeps $\varepsilon \in \{0.1, 0.25, 0.5, 1.0, 2.0\}$. All parameter-sweep runs execute 5,000 steps.

3.3.3 Group 3: Rotating Disk Scaling (Scenario C)

The disk scenario targets large-N scalability and qualitative morphological assessment. Sub-group 3a scales N over $\{10000, 25000, 50000, 75000, 100000\}$ with default parameters and 1,000 steps each, measuring runtime scaling and timing decomposition. Morphological evolution (spiral arm formation, bar instability) is assessed through interactive-mode visualisation at selected timesteps (Figure 7) and reported descriptively.



(a) $t = 0$ (b) $t = 1$

(c) $t = 5$ (d) $t = 10$

Figure 7: Morphological evolution of the rotating exponential disk (Scenario C, $N = 50000$). Panels show the face-on particle distribution at four simulation times, illustrating the development of spiral structure.

3.3.4 Group 4: Direct vs Tree Crossover

To identify the particle count at which hierarchical force evaluation becomes faster than direct $O(N^2)$ summation, this group runs both the direct summation and tree-based force evaluation at $N \in \{100, 200, 500, 1000, 2000, 5000\}$ using the Plummer sphere scenario. Each configuration runs for 500 steps with default physics parameters.

3.3.5 Group 5: Native vs Browser Execution

To test WebGPU's portability promise (RQ3), we run the same GPU LBVH Barnes–Hut configuration from Group 2a in a browser via Emscripten (Emscripten contributors, 2024). The native C++ codebase is cross-compiled to WebAssembly (WASM) with two key flags: `-sASYNCIFY`, which transforms synchronous C++ code into asynchronous form so that the simulation can yield control to the browser's event loop between timesteps (required because browsers do not allow long-running synchronous code on the main thread), and `-sALLOW_MEMORY_GROWTH=1`, which permits the WebAssembly linear memory to grow dynamically as particle count increases rather than requiring a fixed-size allocation at compile time. The resulting WebAssembly module runs in a headless Chromium instance on the same hardware, using the browser's WebGPU implementation (backed by the same Metal driver). Particle counts sweep $N \in$

$\{100, 500, 1000, 2000, 5000, 10000, 25000, 50000, 100000\}$ with all other parameters matching Group 2a ($\Delta t = 0.001$, $\theta = 0.75$, $\varepsilon = 0.5$, leapfrog, GPU LBVH, 1000 steps).

Two timing metrics are collected: (1) wall-clock milliseconds per step, computed from the console-log timestamps emitted every 100 steps, which captures all overhead including event-loop scheduling and Emscripten asyncify yields; and (2) GPU-side timing from the sampled per-step tree/force/integrate breakdowns logged to the console. Energy drift at the final step is compared to the native run at each N to verify numerical consistency across platforms. A GPU command-buffer flush was required after each compute dispatch in the browser path to prevent command coalescing from stalling the pipeline; this fix had no effect on native execution.

3.3.6 Group 6: Cross-Backend Implementation Comparison

To measure the performance impact of the WebGPU implementation layer itself, we run the same simulation across four WebGPU implementations on the same Apple M2, all backed by the Metal graphics API: wgpu-native (the default native backend), Dawn (Google, 2024) (Google's WebGPU implementation), Chrome (browser WebGPU via Emscripten), and Safari (browser WebGPU via Emscripten). This design isolates the overhead introduced by each implementation from the GPU compute itself, since all four paths execute identical WGSL shaders on the same Metal driver. Recent work on WebGPU dispatch overhead has shown that implementation choice within the same backend can produce up to $2.2\times$ variation in per-dispatch cost (Maczan, 2026), making this comparison directly relevant to understanding the platform overhead reported in Group 5. Particle counts sweep $N \in \{1000, 10000, 100000\}$ using the frozen-state benchmarking protocol described above (50 warmup steps, 100 measured steps, positions held constant). For each implementation and N, the mean milliseconds per step, standard deviation, 95% confidence interval, and coefficient of variation are reported.

3.3.7 Group 7: LBVH Construction Pass Breakdown

To identify which phases of the LBVH construction pipeline dominate total tree-build time, per-pass timing is collected at $N \in \{1000, 5000, 10000, 50000, 100000\}$ using CPU-side timing markers placed between each compute dispatch. The six LBVH sub-passes (global AABB reduction, Morton code generation, radix sort, Karras topology construction, leaf initialisation, bottom-up aggregation) and the force evaluation pass are timed individually. This decomposition reveals whether the sort phase, the topology construction, or the bottom-up aggregation is the primary bottleneck, and how the balance shifts with N.

4) Results and Analysis

Results are organised by research question: RQ1 (scalability and pipeline bottlenecks), RQ2 (WebGPU abstraction overhead relative to native Metal), and RQ3 (browser feasibility). All timing values are mean ms/step over 100 measured steps after 50 warmup steps, reported with standard deviation and coefficient of variation. Energy drift is $\Delta E = |E(t) - E(0)| \ / \ |E(0)|$.

4.1 RQ1: Scalability and Pipeline Bottlenecks

4.1.1 Runtime Scaling with Particle Count

Table 6 presents the mean runtime per step and its three-component decomposition for the GPU LBVH Barnes–Hut path across the Plummer sphere scenario. Total runtime increases from 5.9 ms at $N = 1000$ to 180.1 ms at $N = 100000$.

N	ms/step	± std	CV	Tree (ms)	Force (ms)	Integrate (ms)
1,000	5.86	0.57	0.10	0.31	5.52	0.03
5,000	7.35	2.59	0.35	0.25	7.07	0.03
10,000	11.00	2.31	0.21	0.34	10.62	0.04
50,000	65.46	1.82	0.03	0.23	65.20	0.02
100,000	180.11	3.19	0.02	0.28	179.79	0.03

Table 6: Performance summary with timing decomposition (Plummer sphere, leapfrog, $\theta = 0.75$, $\varepsilon = 0.5$, wgpu-native/Metal). Force evaluation dominates at all N, accounting for 94–99% of total step time.

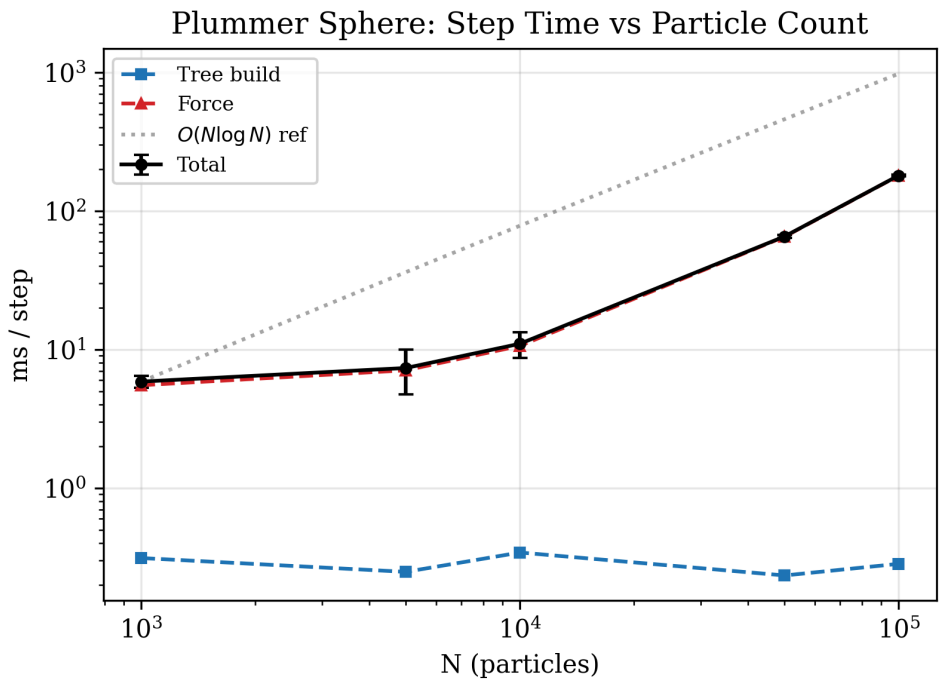


Figure 8: Mean runtime per timestep as a function of N for the Plummer sphere scenario (GPU LBVH, leapfrog, $\theta = 0.75$, wgpu-native/Metal). Total and force curves nearly overlap; tree construction remains flat.

Force evaluation dominates at every particle count — 94% of step time at $N = 1000$, 99.8% at $N = 100000$. Tree construction stays below 0.35 ms even at $N = 100000$. The parallel construction method scales well; any further optimisation effort should go into the BVH traversal shader.

4.1.2 LBVH Construction Breakdown

Table 7 decomposes tree construction into its six individual passes. All six construction passes remain approximately constant with N (1.3–5.1 ms each), confirming that the LBVH pipeline scales well. The radix sort is the most expensive construction pass at 3.5–5.1 ms, accounting for 34–41% of total construction time. Force evaluation, shown in the final column for reference, is the component that drives the overall scaling behaviour.

N	AABB	Morton	Sort	Karras	Leaf	Aggr.	Force
1,000	1.65	1.38	5.05	1.36	1.36	1.39	2.72
5,000	1.37	1.32	4.40	1.29	1.32	1.32	7.59
10,000	1.57	1.42	4.53	1.36	1.39	1.38	13.45
50,000	1.40	1.33	3.54	1.35	1.33	1.32	62.02
100,000	1.33	1.32	4.08	1.31	1.32	1.32	175.39

Table 7: Per-pass timing (ms) for the LBVH construction pipeline and force evaluation. Construction passes are approximately constant with N; force evaluation drives the overall scaling.

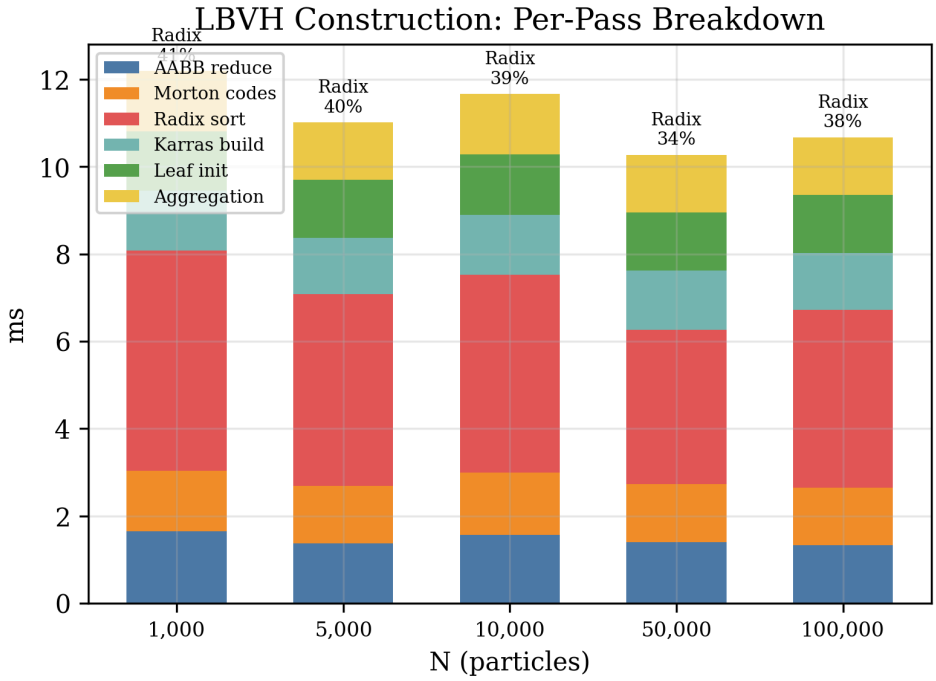


Figure 9: Stacked bar chart of LBVH construction passes at each N. Radix sort is the largest single pass (34–41%). Total construction time is approximately constant across particle counts.

4.1.3 Direct vs Tree: Speed–Accuracy Trade-off

Table 8 compares runtime and energy drift between direct $O(N^2)$ summation and the tree-based $O(N \log N)$ path. Direct summation is faster at all tested particle counts — its regular, branch-free access pattern maps well onto the GPU. But the tree path produces far lower energy drift: at $N = 100000$, direct summation gives $\Delta E = 2.01$ while the tree path achieves $\Delta E = 0.076$. The real crossover is not runtime but *accuracy*: the tree path trades throughput for the force accuracy needed to keep the simulation physically meaningful.

N	Direct (ms)	Tree (ms)	Direct drift	Tree drift
1,000	1.53	5.86	2.25×10^{-2}	6.41×10^{-5}
5,000	4.03	7.35	1.21×10^{-1}	2.50×10^{-4}
10,000	7.46	11.00	2.11×10^{-1}	8.65×10^{-3}
50,000	35.64	65.46	9.88×10^{-1}	6.07×10^{-2}
100,000	140.37	180.11	2.01	7.58×10^{-2}

Table 8: Direct vs tree force evaluation: runtime and energy drift (Plummer sphere, leapfrog, $\theta = 0.75$). Direct summation is faster but accumulates substantially more energy drift, particularly at large N.

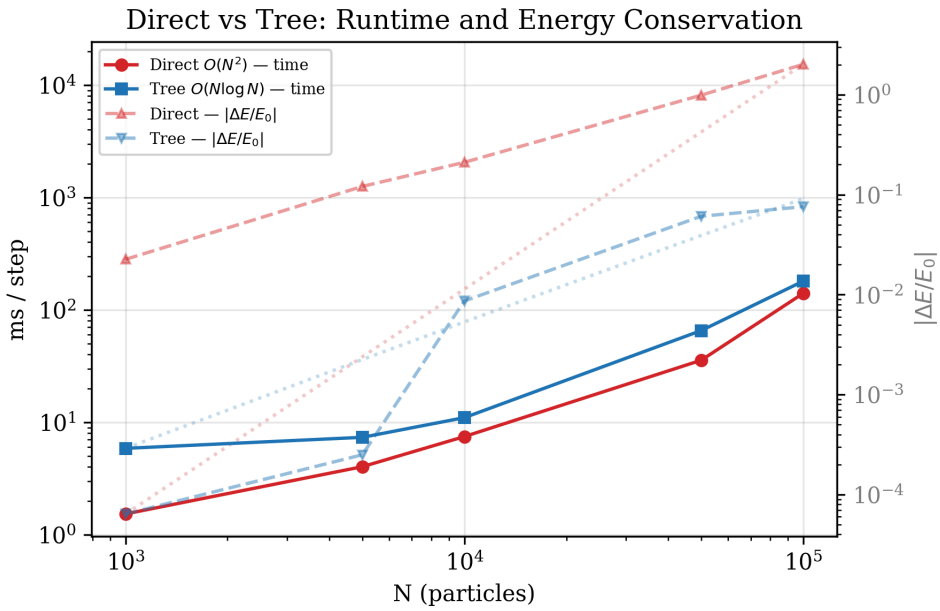


Figure 10: Direct vs tree force evaluation. Left axis: runtime (ms/step). Right axis: final energy drift. The tree path trades throughput for force accuracy.

4.2 RQ2: Abstraction Overhead

4.2.1 WebGPU vs Native Metal

Table 9 compares the WebGPU solver (wgpu-native/Metal) against the native Metal Barnes–Hut baseline (UniSim (Gomes, 2024)) on the same Apple M2 hardware. Both implementations use the same Metal graphics driver; the difference reflects the overhead of the WebGPU abstraction layer.

N	Metal (ms)	WebGPU (ms)	Ratio (WebGPU/Metal)
1,000	2.94	5.86	2.0×
5,000	10.09	7.35	0.73×
10,000	21.35	11.00	0.52×
50,000	121.77	65.46	0.54×
100,000	516.78	180.11	0.35×

Table 9: WebGPU (wgpu-native) vs native Metal (UniSim (Gomes, 2024), stabilised fork (Alsaheb, 2026)) Barnes–Hut performance on the same Apple M2. At $N \geq 5000$ the WebGPU implementation outperforms the Metal baseline, reflecting the efficiency of the fully GPU-resident LBVH pipeline.

At $N = 1000$, WebGPU is 2.0× slower than native Metal, reflecting per-dispatch overhead at small workloads. At $N \geq 5000$, the WebGPU implementation is consistently faster, reaching 2.9× faster at $N = 100000$ (180.11 ms vs 516.78 ms). This advantage reflects differences in tree construction and traversal strategy — the fully GPU-resident LBVH with the optimised traversal shader described in the methodology outperforms UniSim's approach at scale — rather than WebGPU being inherently faster than Metal. The key finding for RQ2 is that the WebGPU abstraction layer imposes no measurable overhead once GPU compute dominates at $N \geq 5000$.

4.2.2 Cross-Backend Implementation Comparison

Table 10 compares four WebGPU implementations on the same hardware, all backed by the Metal API. This isolates the overhead introduced by each implementation from the GPU compute itself (Maczan, 2026).

Implementation	N=1K (ms)	N=10K (ms)	N=100K (ms)	CV (100K)
Dawn	1.40	8.54	272.73	0.01
wgpu-native	5.86	11.00	180.11	0.02
Chrome	4.87	17.81	260.74	0.01
Safari	9.52	20.97	281.59	0.01

Table 10: Per-step runtime across four WebGPU implementations (Plummer sphere, $\theta = 0.75$, frozen-state protocol). All four use the Metal backend on Apple M2. Dawn is fastest at small N; wgpu-native scales best to large N.

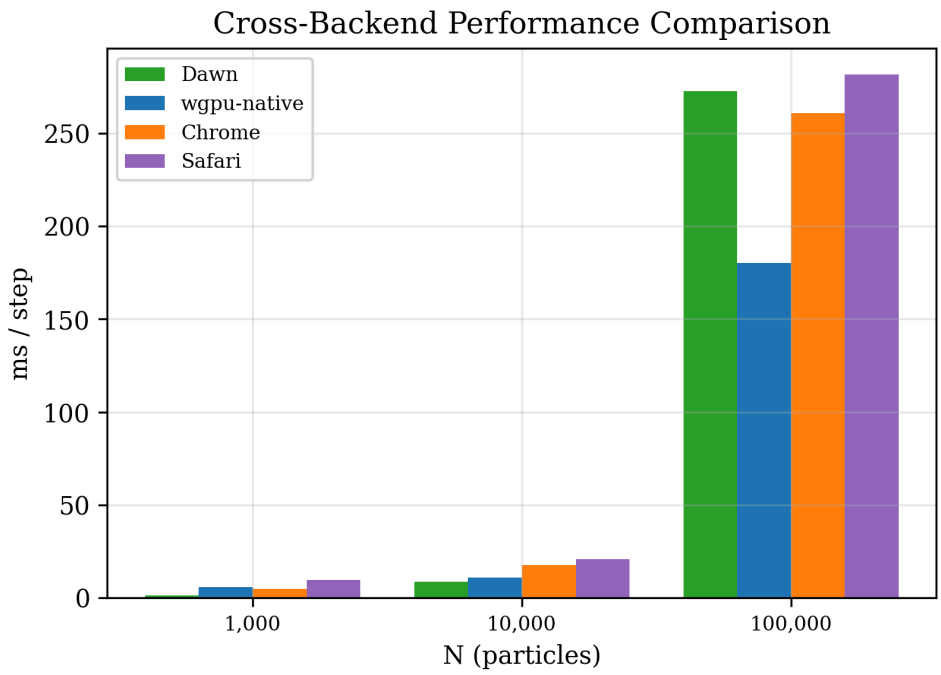


Figure 11: Grouped bar chart of per-step runtime across four WebGPU implementations at three particle counts. Implementation choice produces large performance variation, particularly at $N = 100000$.

The variation across implementations is large. Dawn has the lowest per-dispatch overhead (1.40 ms at $N = 1000$) but scales to 272.73 ms at $N = 100000$; wgpu-native starts higher (5.86 ms) but scales better (180.11 ms). Chrome achieves lower per-dispatch overhead than wgpu-native at small N (4.87 ms vs 5.86 ms at $N = 1000$) but scales worse (260.74 ms at $N = 100000$). Safari is slowest at every N. These findings are consistent with Maczan's observation that implementation choice within the same backend produces significant performance variation (Maczan, 2026).

4.3 RQ3: Browser Feasibility

Table 11 compares native wgpu-native execution against browser execution via Chrome for the same simulation configuration. At $N = 1000$, Chrome is slightly faster than wgpu-native (4.87 ms vs 5.86 ms, overhead $0.8\times$), reflecting lower per-dispatch overhead in the browser's WebGPU implementation at small workloads. As N grows and force evaluation dominates, Chrome's scaling disadvantage becomes apparent: the overhead peaks at $2.0\times$ ($N = 5000$) and then gradually decreases to $1.4\times$ at $N = 100000$ (260.74 ms vs 180.11 ms) as GPU compute time increasingly dominates both platforms.

N	Native (ms)	Chrome (ms)	Overhead	Native drift
1,000	5.86	4.87	0.8×	6.41×10^{-5}
5,000	7.35	14.34	2.0×	2.50×10^{-4}
10,000	11.00	17.81	1.6×	8.65×10^{-3}
50,000	65.46	94.93	1.5×	6.07×10^{-2}
100,000	180.11	260.74	1.4×	7.58×10^{-2}

Table 11: wgpu-native vs Chrome wall-clock ms/step and energy drift. Chrome is faster at $N = 1000$ but scales worse; the overhead converges toward 1.4× at large N.

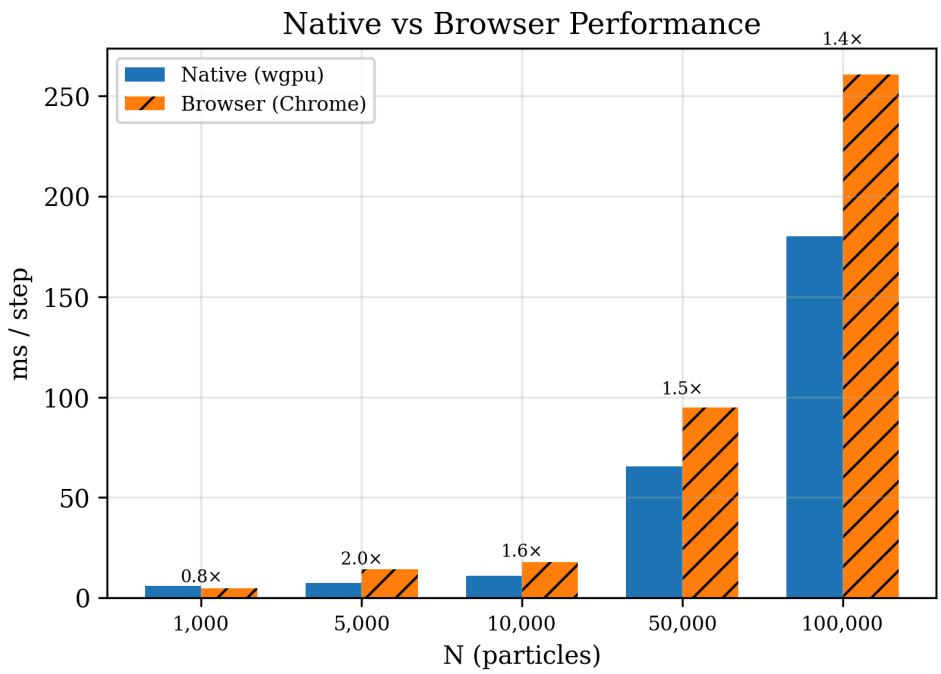


Figure 12: Native vs browser execution time per step. Chrome has lower per-dispatch overhead at small N but scales worse; the overhead ratio narrows as GPU compute dominates.

4.4 Numerical Quality

Energy drift is reported as a secondary observation characterising the precision and approximation quality of the solver. The opening-angle sweep at $N = 5000$ (Table 12) shows that drift increases with θ: from $\Delta E/\ |E(0)| = 2.67 \times 10^{-4}$ at $\theta = 0.3$ to 3.31×10^{-2} at $\theta = 1.0$. Larger opening angles admit more distant nodes into the force approximation, introducing greater truncation error. Runtime decreases modestly with θ (9.30 ms to 6.96 ms), reflecting fewer node interactions at larger opening angles. Momentum is conserved to within 0.1% over 5,000 steps at all tested θ values.

θ	**Mean ms/step**	**Final drift**
0.3	9.30	2.67×10^{-4}
0.5	8.24	2.75×10^{-3}
0.7	7.57	9.77×10^{-3}
1.0	6.96	3.31×10^{-2}

Table 12: Effect of opening angle θ on runtime and energy drift (Plummer sphere, $N = 5000$, 5000 steps). Drift increases with θ, showing the tree approximation contributes to numerical error alongside 32-bit precision.

4.5 Summary of Findings

Force evaluation accounts for 94–99% of step time, with LBVH construction staying below 0.35 ms even at $N = 100000$ (RQ1). The WebGPU abstraction costs 2.0× at $N = 1000$ but the solver outperforms the Metal baseline at $N \geq 5000$, with large variation across WebGPU implementations (RQ2). Chrome is faster than native at $N = 1000$ but scales worse, with the overhead converging to 1.4× at $N = 100000$ (RQ3). Energy drift increases with opening angle θ, ranging from 2.67×10^{-4} to 3.31×10^{-2}, with 32-bit precision and the tree approximation both contributing to numerical error. These findings are interpreted in the following discussion section.

5) Discussion

5.1 Interpretation of Results

5.1.1 Scalability and Pipeline Structure

Total runtime grows from 5.86 ms to 180.11 ms as N increases 100-fold. Force evaluation is responsible for nearly all of it — 94–99% of step time. At small N the GPU is underutilised and dispatch overhead dominates; as N grows, the parallel hardware absorbs the traversal workload. Tree construction, by contrast, stays below 0.35 ms at all tested particle counts, with individual passes roughly constant in N. The message is clear: optimisation effort belongs in the BVH traversal shader, not the tree-build pipeline. The direct-versus-tree comparison reinforces this. Direct summation is faster at every tested N — its regular, branch-free access pattern maps well onto the GPU — but the tree path produces 26× lower energy drift at $N = 100000$. For any simulation that needs to remain physically meaningful over many timesteps, the hierarchical approach is not optional.

5.1.2 Abstraction Overhead

That the WebGPU solver outperforms the native Metal baseline (UniSim (Gomes, 2024)) at $N \geq 5000$ deserves careful interpretation. We do not claim WebGPU is inherently faster than Metal. The advantage comes from the fully GPU-resident LBVH pipeline — radix sort, Karras topology construction, and bottom-up aggregation, all on-device — which avoids the CPU–GPU coordination overhead in UniSim's tree construction. The gap widens with N (0.52× at $N = 10000$, 0.35× at $N = 100000$), confirming that the benefit is structural, not incidental. The cross-backend comparison tells a different story: implementation choice within the same Metal backend produces large performance variation. Dawn has the lowest per-dispatch overhead (1.40 ms at $N = 1000$) but scales to 272.73 ms at $N = 100000$; wgpu-native starts higher but reaches only 180.11 ms. This matches Maczan's finding that WebGPU implementation choice alone can produce up to 2.2× variation in per-dispatch cost (Maczan, 2026). For compute-heavy workloads, wgpu-native is the better choice at large N; Dawn is preferable for low-latency small dispatches.

5.1.3 Browser Platform Viability

The browser overhead pattern is not a simple fixed cost. At $N = 1000$, Chrome is actually faster than wgpu-native (4.87 ms vs 5.86 ms, 0.8×), suggesting lower per-dispatch overhead in the browser's WebGPU path. But as force evaluation dominates at larger N, Chrome scales less efficiently: the overhead peaks at 2.0× ($N = 5000$) then gradually narrows to 1.4× at $N = 100000$ (260.74 ms vs 180.11 ms). This pattern — browser faster at small N, slower at large N — implies a scaling difference in how the browser's WebGPU implementation handles the

irregular memory access patterns of BVH traversal, rather than a fixed event-loop penalty. The practical implication is that browser deployment is viable for interactive N-body simulation at moderate particle counts, with an overhead of roughly 1.4–1.5$\times$ at scientifically meaningful N ($\geq$ 50000) — subject to the precision constraints discussed below (Sengupta et al., 2025).

5.1.4 Numerical Quality

The theta-sweep results show a clear dependence of energy drift on opening angle: $\Delta E/\ |E(0)|$ increases from 2.67×10^{-4} at $\theta = 0.3$ to 3.31×10^{-2} at $\theta = 1.0$, spanning two orders of magnitude. This is the expected physical behaviour — larger θ admits more distant nodes into the force approximation, introducing greater truncation error. The implication is that both the tree approximation and 32-bit floating-point precision contribute to numerical error; the opening angle is not irrelevant. For practical use, the leapfrog integrator remains the correct choice: it conserves momentum to within 0.1% over 5,000 steps and maintains stable orbits, consistent with its theoretical properties (Binney & Tremaine, 2008; Springel, 2005). We note that the observed Plummer sphere expansion is physical, not numerical — it is a known behaviour of finite-N systems with approximate forces (Binney & Tremaine, 2008).

5.2 Comparison with Published Results

Direct comparison with CUDA-based Barnes–Hut implementations is complicated by differences in hardware generation, floating-point precision (64-bit in CUDA studies vs 32-bit here), tree topology (octree vs BVH), and opening criteria. Still, several structural comparisons are worth making.

Burtscher and Pingali report roughly 10 billion body-body interactions per second for CUDA Barnes–Hut on an NVIDIA GTX 280 (Burtscher & Pingali, 2011) — about 10 ms per step at $N = 100000$ with $\theta = 0.75$ on 2010-era hardware. Our WebGPU implementation needs 180.11 ms at the same N, reflecting the BVH traversal shader (99% of step time) and the irregular memory access patterns of LBVH vs octree traversal. A cross-generation, cross-topology comparison like this is not apples-to-apples, but it does confirm that the WebGPU compute shader model can support a complete solver pipeline at large N, with throughput limited by traversal efficiency rather than platform overhead.

Gaburov, Bédorf, and Portegies Zwart report tree-construction overhead of 15–25% of total step time in their CUDA octree implementation (Gaburov et al., 2010). In our case, LBVH construction accounts for less than 0.2% (0.28 ms out of 180.11 ms at $N = 100000$). The difference comes down to where the tree is built: we construct the entire LBVH on the GPU — radix sort, topology, bottom-up aggregation — and never transfer tree data across the CPU–GPU boundary.

Nyland, Harris, and Prins report direct $O(N^2)$ GPU throughput of 10–30 GFLOP/s on contemporary NVIDIA hardware (Nyland et al., 2009). Our WebGPU direct-summation path achieves 4.03 ms per step at $N = 5000$ ($\approx 6.2 \times 10^9$ interactions per second), which is competitive given the M2′s integrated GPU and 32-bit precision.

5.3 Limitations

Several limitations bound the external validity of these results.

The most significant is that all experiments ran on a single Apple M2 system. The cross-backend comparison (Group 6) shows that software-side variation across four WebGPU implementations is large, but all four use the same Metal driver on the same hardware. GPU performance varies across vendors. The M2 is an integrated GPU with unified memory — it avoids the CPU–GPU transfer costs of discrete systems, but its compute unit count is modest compared to dedicated NVIDIA or AMD GPUs. The timing results and scaling behaviour we report may not generalise to other hardware or GPU backends (Vulkan, Direct3D 12).

A second constraint is WebGPU's limitation to 32-bit floating-point arithmetic in compute shaders. Together with the tree approximation (controlled by θ), single-precision arithmetic constrains numerical fidelity. The theta sweep shows that both factors contribute: drift varies two orders of magnitude across θ values, while 32-bit precision sets a floor visible in the direct-summation drift ($\Delta E = 2.01$ at $N = 100000$ even without tree approximation). Relatedly, we use only the monopole term of the multipole expansion; quadrupole and higher-order terms, standard in production codes like GADGET-2 (Springel, 2005), would reduce force approximation error at a given θ and potentially allow larger opening angles for the same accuracy.

On the methodology side, we use a global fixed timestep, whereas production codes typically assign each particle an individual timestep proportional to its local dynamical timescale (Springel, 2005). Without adaptive timestepping, the global Δt must be conservative enough for the densest region, which wastes work on the majority of particles. Energy drift as a quantitative metric is also limited to $N \leq 5000$, where direct pair summation for potential energy is still feasible; at larger N we rely on kinetic energy trends and momentum conservation as indirect quality indicators.

6) Future Work

The limitations discussed above point to several concrete next steps, each tied to a specific performance or capability goal.

6.1 Adaptive Individual Timestepping

Our fixed global timestep forces a conservative Δt that accommodates the densest region of the simulation. Adaptive individual timestepping, as implemented in GADGET-2 (Springel, 2005), assigns each particle a timestep proportional to its local dynamical timescale ($\sqrt{\frac{\varepsilon}{|a_i|}}$) and synchronises particles via block timestepping. Implementing this in WebGPU compute shaders would require per-particle step bookkeeping in storage buffers and more complex dispatch logic, but it would improve accuracy in dense cores (Plummer spheres, disk centres) without penalising particles in diffuse outer regions.

6.2 Cross-Hardware and Cross-Browser Benchmarking

Our cross-backend comparison (Group 6) tested four WebGPU implementations on the same Apple M2 and found large cross-implementation variation, but all four use the same Metal driver. Extending the benchmarks to non-Apple hardware — discrete NVIDIA and AMD GPUs using the Vulkan backend, and Intel Arc using Direct3D 12 — would determine whether the scaling characteristics and relative implementation rankings observed here generalise across GPU architectures. Firefox (once WebGPU is enabled) represents another untested browser implementation. Mobile devices (smartphones and tablets), where GPU scheduling, thermal throttling, and memory bandwidth differ from desktop systems, are a further priority: the browser scaling overhead observed here ($1.4\times$ at $N = 100000$) may vary on devices with different GPU scheduling characteristics. Such testing would extend the findings of Sengupta et al. (Sengupta et al., 2025) to hierarchical N-body simulation across diverse consumer hardware.

6.3 Higher-Order Multipole Expansion

Extending the monopole approximation to include quadrupole moments in the BVH nodes would reduce the force approximation error at a given θ, potentially allowing the use of larger opening angles (and correspondingly faster traversal) for the same accuracy target. The additional per-node storage (a symmetric 3×3 tensor per node) fits within WebGPU storage buffer limits, and the extra arithmetic per interaction is modest. This extension would bring the force accuracy closer to production codes and enable more meaningful comparison with GADGET-2′s multipole expansion (Springel, 2005).

6.4 64-Bit Precision When Available

Our results show that 32-bit GPU arithmetic is the dominant constraint on numerical quality. As the WebGPU specification evolves, optional 64-bit floating-point support in compute shaders (`f64`) may become available on supporting hardware. Benchmarking the precision-versus-performance trade-off with 64-bit force evaluation would determine whether the energy conservation improvement justifies the throughput reduction, and would enable direct integrator-quality comparisons (leapfrog vs Euler) that are currently confounded by the precision difference between GPU and CPU paths.

6.5 Multi-Component Galaxy Models

Scenario C uses a simplified single-component disk. Adding a central bulge, an exponential disk, and a dark matter halo following standard multi-component galaxy models (Binney & Tremaine, 2008) would enable comparison with observational rotation curves and provide a more physically realistic testbed. The simulation architecture already supports heterogeneous particle masses, so accommodating multiple components would require changes mainly in the initial-condition generator.

7) Conclusion

We presented a hierarchical N-body solver for galactic dynamics, implemented entirely in WebGPU, with a fully GPU-resident Linear Bounding Volume Hierarchy (LBVH) constructed and traversed on-device each timestep. We evaluated it across three benchmark scenarios (two-body orbit, Plummer sphere, and rotating exponential disk) with systematic parameter sweeps targeting three research questions: scalability and pipeline bottlenecks, WebGPU abstraction overhead relative to native Metal, and browser feasibility. Numerical quality under 32-bit precision was characterised as a secondary observation throughout.

7.1 Scalability and Pipeline Bottlenecks (RQ1)

Runtime grows from 5.86 ms per step at $N = 1000$ to 180.11 ms at $N = 100000$, with force evaluation accounting for 94–99% of step time. The LBVH construction pipeline stays below 0.35 ms even at $N = 100000$, with individual passes roughly constant in N — the BVH traversal shader is clearly the primary optimisation target. Direct $O(N^2)$ summation is faster at every tested particle count, but it accumulates far more energy drift: $\Delta E = 2.01$ vs 0.076 for the tree path at $N = 100000$. The hierarchical approach trades throughput for the force accuracy needed to keep simulations physically meaningful.

7.2 Abstraction Overhead (RQ2)

Against a native Metal Barnes–Hut baseline (UniSim (Gomes, 2024)) on the same Apple M2, the WebGPU layer costs 2.0× at $N = 1000$, where per-dispatch overhead dominates. At $N \geq 5000$ the WebGPU solver is consistently faster — 2.9× at $N = 100000$ (180.11 ms vs 516.78 ms) — because the fully GPU-resident LBVH pipeline eliminates CPU–GPU coordination, not because WebGPU is inherently faster than Metal. A four-way comparison across WebGPU implementations (wgpu-native, Dawn, Chrome, Safari) shows large cross-implementation variation, consistent with Maczan (Maczan, 2026): Dawn has the lowest per-dispatch overhead at small N; wgpu-native scales best to large N.

7.3 Browser Feasibility (RQ3)

Chrome's browser WebGPU implementation achieves lower per-dispatch overhead than wgpu-native at small N (4.87 ms vs 5.86 ms at $N = 1000$, overhead 0.8×). As force evaluation dominates at larger particle counts, Chrome scales less efficiently: the overhead peaks at 2.0× ($N = 5000$) and then gradually decreases to 1.4× at $N = 100000$ (260.74 ms vs 180.11 ms). The overhead narrows with N but does not vanish, indicating a scaling rather than a fixed-cost difference between platforms. Nevertheless, the browser sustains the same qualitative scaling

behaviour as native execution, supporting WebGPU's viability for browser-based scientific simulation.

7.4 Contribution

Taken together, the results show that WebGPU's compute shader model is sufficient for a complete scientific computing workload — parallel tree construction, hierarchical force evaluation, and symplectic integration — benchmarked across four implementations and compared against a native Metal baseline. The solver matches or exceeds native Metal performance at scientifically relevant particle counts, and browser execution remains within $1.4\times$ of native throughput at $N = 100000$. By building from a single C++/WGSL codebase targeting both desktop and browser, we show that accessible, interactive galactic dynamics simulation is achievable without specialised hardware or vendor-locked APIs. The performance data and identified precision limitations reported here should serve as a reference point for future GPU-accelerated scientific computing in WebGPU as the standard and its implementations mature.

8) Appendix

H.1 Source Code Repositories

The source code for all software used in this work is publicly available:

Repository	URL
N-body solver (this work)	`https://github.com/haxybaxy/webgpu-galaxy`
UniSim fork (Metal baseline)	`https://github.com/haxybaxy/unisim`
Paper source (Typst)	`https://github.com/haxybaxy/capstone`

Table 13: Source code repositories.

H.2 Command-Line Interface Reference

The solver is invoked as a single binary with all parameters specified via command-line flags. Table 14 lists the available options.

Flag	Type	Default	Description
`--scenario`	`twobody` | `plummer` | `disk`	`disk`	Benchmark scenario
`--N`	integer	`10000`	Particle count (ignored for twobody)
`--dt`	float	`0.001`	Timestep Δt
`--theta`	float	`0.75`	Opening angle θ
`--softening`	float	`0.5`	Plummer softening ε
`--seed`	integer	`42`	Random seed for initial conditions
`--steps`	integer	0 (interactive)	Step count; required for headless
`--force-method`	`tree` | `direct`	`tree`	Force evaluation method
`--export`	path	(none)	CSV output file path
`--headless`	flag	off	Batch mode with no window
`--sync-timing`	flag	off	GPU fence before each timestamp
`--benchmark-passes`	flag	off	Per-pass LBVH timing (implies sync)

Table 14: Command-line flags. All flags are optional; unspecified parameters use the defaults shown.

H.3 Experiment Invocations

Below are representative invocations for each experiment group. In all cases, the swept parameter is varied while other flags remain as shown. The benchmarking protocol (50 warmup steps discarded, 100 measured) is applied during post-processing of the CSV output; the `--steps` flag is set to 150 accordingly.

H.3.1 Group 1: Two-Body Validation

```
./galaxysim --scenario twobody --dt 0.001 --theta 0.75 \
  --softening 0.5 --seed 42 --steps 50000 --headless \
  --sync-timing --export results/A_leapfrog_dt001.csv
```

Swept: `--dt` over $\{0.0001, 0.0005, 0.001, 0.005, 0.01\}$. Particle count is fixed at $N = 2$ by the scenario.

H.3.2 Group 2a: N-Scaling (Plummer)

```
./galaxysim --scenario plummer --N 1000 --dt 0.001 --theta 0.75 \
  --softening 0.5 --seed 42 --steps 150 --headless \
  --sync-timing --export results/B_scale_N1000.csv
```

Swept: `--N` over $\{1000, 5000, 10000, 50000, 100000\}$.

H.3.3 Groups 2b/2c/2e: Parameter Sweeps (Plummer, $N = 5000$)

```
./galaxysim --scenario plummer --N 5000 --dt 0.001 --theta 0.3 \
  --softening 0.5 --seed 42 --steps 5000 --headless \
  --sync-timing --export results/B_theta03.csv
```

2b swept: `--theta` over $\{0.3, 0.5, 0.7, 1.0\}$.

2c swept: `--dt` over $\{0.00005, 0.0001, 0.0005, 0.001, 0.005\}$.

2e swept: `--softening` over $\{0.1, 0.25, 0.5, 1.0, 2.0\}$.

H.3.4 Group 3a: Disk N-Scaling

```
./galaxysim --scenario disk --N 10000 --dt 0.001 --theta 0.75 \
  --softening 0.5 --seed 42 --steps 150 --headless \
  --sync-timing --export results/C_scale_N10000.csv
```

Swept: `--N` over $\{10000, 25000, 50000, 75000, 100000\}$.

H.3.5 Group 4: Direct vs Tree Crossover

```
./galaxysim --scenario plummer --N 1000 --dt 0.001 --theta 0.75 \
  --softening 0.5 --seed 42 --steps 150 --headless \
  --sync-timing --force-method direct \
  --export results/D_direct_N1000.csv
```

Swept: `--N` over $\{1000, 5000, 10000, 50000, 100000\}$, each run twice with `--force-method tree` and `--force-method direct`.

H.3.6 Group 5: Native vs Browser

Native runs use the same invocation as Group 2a. Browser runs use the Emscripten-compiled WebAssembly build served to a headless Chromium instance; timing is extracted from console-log timestamps.

H.3.7 Group 6: Cross-Backend Comparison

```
./galaxysim --scenario plummer --N 1000 --dt 0.001 --theta 0.75 \
  --softening 0.5 --seed 42 --steps 150 --headless \
  --sync-timing --export results/F_wgpu_N1000.csv
```

Swept: `--N` over {1000, 10000, 100000}. Repeated for four backends: wgpu-native, Dawn, Chrome (Emscripten), Safari (Emscripten). Positions are held frozen (forces computed but not applied) to isolate dispatch overhead.

H.3.8 Group 7: LBVH Pass Breakdown

```
./galaxysim --scenario plummer --N 1000 --dt 0.001 --theta 0.75 \
  --softening 0.5 --seed 42 --steps 150 --headless \
  --benchmark-passes --export results/G_lbvh_N1000.csv
```

Swept: `--N` over {1000, 5000, 10000, 50000, 100000}. The `--benchmark-passes` flag enables per-pass timing.

H.4 Complete Experiment Parameter Table

Table 15 lists every distinct experiment configuration. All runs use leapfrog integration and seed 42 unless noted.

Group	Scenario	N	Δt	θ	ε	Force	Steps
1	twobody	2	0.0001–0.01	0.75	0.5	tree	50,000
2a	plummer	1K–100K	0.001	0.75	0.5	tree	150
2b	plummer	5,000	0.001	0.3–1.0	0.5	tree	5,000
2c	plummer	5,000	5e-5–5e-3	0.75	0.5	tree	5,000
2e	plummer	5,000	0.001	0.75	0.1–2.0	tree	5,000
3a	disk	10K–100K	0.001	0.75	0.5	tree	150
4	plummer	1K–100K	0.001	0.75	0.5	both	150
5	plummer	1K–100K	0.001	0.75	0.5	tree	150
6	plummer	1K–100K	0.001	0.75	0.5	tree	150
7	plummer	1K–100K	0.001	0.75	0.5	tree	150

Table 15: Complete parameter table for all experiment groups. Ranges indicate the swept variable; all other parameters are held fixed. Group 4 runs both force methods at each N. Groups 5 and 6 repeat across execution platforms (see text). All runs use seed 42 and leapfrog integration.

References

Aarseth, S., Henon, M., & Wielen, R. (1974). A Comparison of Numerical Methods for the Study of Star Cluster Dynamics. *\aap*, *37*(1), 183–187.

Alsaheb, Z. (2026,). *UniSim fork: stabilised for large-N benchmarking*. https://github.com/haxybaxy/unisim

Apple Inc. (2024,). *Metal: Apple's low-level graphics and compute API*. https://developer.apple.com/metal/

Barnes, J., & Hut, P. (1986). A hierarchical O(N log N) force-calculation algorithm. *Nature*, *324*(6096), 446–449. https://doi.org/10.1038/324446a0

Batcher, K. E. (1968). Sorting Networks and Their Applications. *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*, 307–314. https://doi.org/10.1145/1468075.1468121

Binney, J., & Tremaine, S. (2008). *Galactic Dynamics: Second Edition* (REV - Revised, 2). Princeton University Press. http://www.jstor.org/stable/j.ctvc778ff

Burtscher, M., & Pingali, K. (2011). An Efficient CUDA Implementation of the Tree-Based Barnes Hut n-Body Algorithm. *GPU Computing Gems Emerald Edition*, . https://doi.org/10.1016/B978-0-12-384988-5.00006-1

Congote, J., Segura, A., Kabongo, L., Moreno, A., Posada, J., & Ruiz, O. (2011). Interactive visualization of volumetric data with WebGL in real-time. *Proceedings of the 16th International Conference on 3d Web Technology*, 137–146. https://doi.org/10.1145/2010425.2010449

Emscripten contributors. (2024,). *Emscripten: An LLVM-to-WebAssembly compiler*. https://emscripten.org/

Fluke, C. J., Barnes, D. G., Barsdell, B. R., & Hassan, A. H. (2011). Astrophysical Supercomputing with GPUs: Critical Decisions for Early Adopters. *Publications of the Astronomical Society of Australia*, *28*(1), 15–27. https://doi.org/10.1071/as10019

Freeman, K. C. (1970). On the Disks of Spiral and S0 Galaxies. *The Astrophysical Journal*, *160*, 811. https://doi.org/10.1086/150474

Gaburov, E., Bédorf, J., & Portegies Zwart, S. (2010). Gravitational tree-code on graphics processing units: Implementation in CUDA. *Procedia Computer Science*, *324*, 1119–1127. https://doi.org/10.1016/j.procs.2010.04.124

gfx-rs contributors. (2024,). *wgpu: Safe and portable GPU abstraction in Rust*. https://github.com/gfx-rs/wgpu

Gomes, F. C. (2024,). *UniSim: Barnes–Hut N-body simulation in Metal*. https://github.com/fercgomes/unisim

Google. (2024,). *Dawn: Google's open-source WebGPU implementation*. https://dawn.googlesource.com/dawn

GPU for the Web Working Group. (2026,). *WebGPU (GPU for the Web) Specification*. https://gpuweb.github.io/gpuweb/

Karras, T. (2012). Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. *Proceedings of the Fourth ACM SIGGRAPH / Eurographics Conference on High-Performance Graphics*, 33–37.

Khronos Group. (2024b,). *Vulkan Specification*. https://www.vulkan.org/

Khronos Group. (2024a,). *WebGPU Specification*. https://www.w3.org/TR/webgpu/

Kreyszig, E. (2011). *Advanced Engineering Mathematics* (10th ed.). Wiley.

LF, G., & Rokhlin, V. (2001). A Fast Algorithm for Particle Simulation. *Journal of Computational Physics*, *73*, 325–348. https://doi.org/10.1016/0021-9991(87)90140-9

Maczan, J. (2026). *Characterizing WebGPU Dispatch Overhead for LLM Inference Across Four GPU Vendors, Three Backends, and Three Browsers*.

Morton, G. M. (1966). *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing*.

Nyland, L., Harris, M., & Prins, J. (2009). Fast N-body simulation with CUDA. *GPU Gem, Vol. 3*, 677–695.

Owens, J. D., Luebke, D., Govindaraju, N., Harris, M., Krüger, J., Lefohn, A. E., & Purcell, T. J. (2007). A Survey of General-Purpose Computation on Graphics Hardware. *Computer Graphics Forum*, *26*(1), 80–113. https://doi.org/https://doi.org/10.1111/j.1467-8659.2007.01012.x

Owens, J., Houston, M., Luebke, D., Green, S., Stone, J., & Phillips, J. (2008). GPU computing. *Proceedings of the IEEE*, *96*, 879–899. https://doi.org/10.1109/JPROC.2008.917757

Peláez, R. P. (2022). *Complex Fluids in the GPU Era: Algorithms and Simulations*. https://raw.githubusercontent.com/RaulPPelaez/tesis/main/manuscript.pdf

Plummer, H. C. (1911). On the Problem of Distribution in Globular Star Clusters. *Monthly Notices of the Royal Astronomical Society*, *71*(5), 460–470. https://doi.org/10.1093/mnras/71.5.460

Salmon, J. K., & Warren, M. S. (1994). Skeletons from the Treecode Closet. *Journal of Computational Physics*, *111*(1), 136–155. https://doi.org/https://doi.org/10.1006/jcph.1994.1050

Sengupta, S., Wu, N., Varvello, M., Jana, K., Chen, S., & Han, B. (2025). From WebGL to WebGPU: A Reality Check of Browser-Based GPU Acceleration. *Proceedings of the 2025 ACM Internet Measurement Conference*, 1018–1024. https://doi.org/10.1145/3730567.3764504

Springel, V. (2005). The cosmological simulation code gadget-2. *Monthly Notices of the Royal Astronomical Society*, *364*(4), 1105–1134. https://doi.org/10.1111/j.1365-2966.2005.09655.x

Sung, N.-J., Ma, J., Kim, T., Choi, Y.-j., Choi, M.-H., & Hong, M. (2025,). *Real-Time Cloth Simulation Using WebGPU: Evaluating Limits of High-Resolution*. https://arxiv.org/abs/2507.11794

Sutter, H. (2005). The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software. *Dr. Dobb's Journal*, *30*(3), 202–210. http://www.gotw.ca/publications/concurrency-ddj.htm

Usta, Z. (2024). WEBGPU: A NEW GRAPHIC API FOR 3D WEBGIS APPLICATIONS. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, 377–382. https://doi.org/10.5194/isprs-archives-XLVIII-4-W9-2024-377-2024

Verlet, L. (1967). Computer ``Experiments'' on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules. *Physical Review*, *159*(1), 98–103. https://doi.org/10.1103/PhysRev.159.98

Wang, Q. (2021). A hybrid Fast Multipole Method for cosmological N-body simulations. *Research in Astronomy and Astrophysics*, *21*(1), 3. https://doi.org/10.1088/1674-4527/21/1/3

Zwart, S. F. P., Belleman, R. G., & Geldof, P. M. (2007). High-performance direct gravitational N-body simulations on graphics processing units. *New Astronomy*, *12*(8), 641–650. https://doi.org/https://doi.org/10.1016/j.newast.2007.05.004